Analytical Modeling of Wind Turbine Dynamics Using Symbolic Lagrangian Mechanics

Dinor Nallbani

Chair: Emmanuel Branlard

Date: Thursday 14th May, 2026

ABSTRACT

The size and flexibility of modern offshore wind turbines lead to new challenges in terms of aeroelastic phenomena threatening structural integrity. This thesis hopes to establish a series of analytical wind turbine models of increasing complexity to study the fundamental trends that governing how system frequencies and aerodynamic damping vary with rotor rotational speed and ambient wind velocity. The research aims to find the physical reasons behind non-intuitive dynamic behavior, such as why the second tower mode varies significantly depending on whether the rotor is modeled as stiff or flexible, and how gyroscopic effects influence the frequencies of the turbine's yawing and tilting modes.

The equations of motion for the multi-body system are derived using the energy-based Lagrange method. For more complex models, the Python package `sympy` is used as a symbolic framework to to systematically generate, manipulate, and linearize these equations. By performing an eigenvalue analysis on the resulting linear systems, the frequency and damping characteristics can be determined. The analytical and numerical outputs of these reduced-order models will be compared against more advanced, high-fidelity numerical tools, providing a transparent and validated understanding of wind turbine aeroelasticity.

CONTENTS

I Introduction 4

I-A Motivation and the Shift to Offshore Wind . . . 4

I-B Problem Statement: Higher-Order Modes and Analytical Challenges . . . 4

I-C Research Objectives and Methodological Approach . . . 5

II Wind Turbine Modeling 5

II-A Structural Dynamics and Symbolic Lagrangian Mechanics . . . 5

III Literature Conclusions 6

III-A Modeling Approaches . . . 7

III-B Aeroelastic Stability, Damping, and Whirling Modes . . . 7

III-C Shape Functions and Finite Element Methods . . . 8

IV Project Evaluation and Management 10

IV-A Evaluation . . . 10

IV-B Communication and Workflow . . . 10

IV-C Project Timeline . . . 10

V Methodology 10

V-A Symbolic Derivation Pipeline . . . 10

V-B Linearization Procedure . . . 11

V-C Eigenvalue Analysis . . . 11

VI Analysis and Results 12

VI-A Tilting Nacelle, Rigid Blade . . . 12

VI-A1 Model Description and Degrees of Freedom . . . 12

VI-A2 Nonlinear Equations of Motion . . . 14

VI-A3 Nonlinear System Response . . . 14

VI-A4 Linearization and Eigenvalue Analysis . . . 15

VI-A5 Linearized System Simulation . . . 17

VI-A6 Validation: Nonlinear vs. Linear Model . . . 18

VI-A7 Key Findings from Model 1 . . . 18

VI-B Tilting Nacelle, Flexible Blade . . . 20

VI-B1 Model Description and Degrees of Freedom . . . 20

VI-B2 Nonlinear Equations of Motion . . . 21

VI-B3 Nonlinear System Response . . . 22

VI-B4 Linearization and Eigenvalue Analysis . . . 23
VI-B5 Linearized System Simulation . . . 25
VI-B6 Validation: Nonlinear vs. Linear Model . . . 26
VI-B7 Key Findings from Model 2 . . . 26

VII Discussion 27
VII-A The Impact of Structural Flexibility on Tower Modes . . . 27
VII-B Gyroscopic Coupling and Energy Transfer . . . 28
VII-C Limitations of the Analytical Approach . . . 28

VIII Conclusion 29

Appendix 30
A Simple Pendulum . . . 30
B Double Pendulum . . . 30
C 2-DOF Rigid Nacelle-Rotor Model . . . 30
D Fixed Nacelle, Flexible Blade . . . 31
E Flexible Tower, Rigid Blade . . . 32
F Rigid Rotating Blade, Tilting Nacelle code . . . 34
G Flexible Rotating Blade, Tilting Nacelle code . . . 43

References 62

I. Introduction

A. Motivation and the Shift to Offshore Wind

The global transition toward sustainable energy systems has placed wind energy among the leading technologies for power generation. To meet growing energy demands and maximize efficiency, the industry has shifted begun shifting its focus from land-based systems to offshore environments. This transition is driven by their physical advantages; specifically, offshore wind turbines offer an opportunity to move sustainable energy generation to locations where higher energy outputs are possible, as "Wind velocity is higher and more dependable at offshore locations" and "offshore wind energy [being] characterized by higher power density" [1]. To effectively harness this energy, offshore wind turbines are larger in size and complexity compared to their onshore counterparts. Understanding their dynamic behavior of larger turbines becomes increasingly important for ensuring structural integrity and operational efficiency. This literature review aims to establish a foundation for using Lagragian mechanics to derive equations of motion (EOMs) investigate the aeroelastic stability of wind turbines and how they vary with rotational and wind speeds.

This drive for offshore deployment is underscored by fundamental physical principles. The available power in the wind, P, increases dramatically with the wind velocity, v:

$$P \propto v^3 \tag{1}$$

Furthermore, the power captured by a turbine is directly proportional to the swept area of its rotor, which scales with the square of the blade length, r:

$$A \propto r^2 \tag{2}$$

As a result, small increases in wind speed or blade length yield large gains in power output. Modern offshore turbines feature extraordinarily long blades and towering support structures to maximize energy capture; however, this unprecedented geometric upscaling introduces an engineering challenge in modeling their dynamic behavior. Today's multi-megawatt turbines no longer behave as rigid bodies, rather operating as highly flexible structures. This increased flexibility introduces complex aeroelastic phenomena — interactions between aerodynamic forces and structural motion — not present in rigid systems. Ensuring their stability of these structures against phenomena such as classical flutter and stall-induced vibrations, is a primary design constraint [2].

B. Problem Statement: Higher-Order Modes and Analytical Challenges

While the first-order modes of wind turbines (such as the first fore-aft tower bending or first flapwise blade bending) are well-documented and standard in design load cases, higher-order modes remain an important area of study which can impact turbine stability. As turbines become larger for offshore purposes, they also lose rigidity, causing modes like the second tower mode to gain prominence in influencing the dynamic response of the structure. The literature identifies that these higher-order modes can exhibit very low or even negative aerodynamic damping under specific operational conditions, leading to potential aeroelastic instabilities such as stall-induced vibrations or classical flutter [2].

The damping mechanisms for the second tower mode are often non-intuitive — defying ”common sense” or classic newtonian expectations. They mechanisms arise from complex aeroelastic couplings with the rotor that vary non-linearly with rotational speed and pitch settings [2]. The continuous rotation of the massive turbine rotor introduces gyroscopic couplings fundamentally altering the system's dynamic response, heavily influencing the frequencies of the turbine's yawing and tilting modes and causing frequency splitting [3].

Currently, the wind energy industry relies heavily on sophisticated, high-fidelity numerical simulators (such as OpenFAST) to analyze these modes. While these tools are highly accurate, these tools often obscure the underlying mathematical relationships governing the system's behavior. They function essentially as numerical black boxes, with parameters going in and results coming out. This makes it difficult to identify the root physical causes of instability when a structural frequency jumps or damping goes negative during a simulation.

C. Research Objectives and Methodological Approach

The primary objective of this thesis is to systematically establish reduced-order, analytical wind turbine models of increasing complexity to uncover the fundamental physical trends governing aeroelastic stability. Rather than relying solely on black-box simulations, this research aims to provide explicit, mathematically transparent explanations for non-intuitive phenomena, such as why the second tower mode varies so significantly depending on whether the rotor is modeled as stiff or flexible, and how gyroscopic effects dynamically dictate the frequencies of the yawing and tilting modes.

To accurately describe the motion of these complex systems, this work relies on Lagrangian mechanics. This energy-based methodology utilizes the principle of least action to analyze complex, rotating reference frames much more efficiently than the basic $F = ma$ approach used in Newtonian mechanics [4]. To bridge the gap between continuous flexible structures and discrete mathematics, this project employs the Rayleigh-Ritz method to map “the infinite number of DOF necessary to describe a flexible body onto a reduced and converging set of shape functions” [5].

Because deriving equations of motion manually becomes intractable for flexible, rotating systems, the literature emphasizes the utility of symbolic computational frameworks over purely numerical solvers. The Python package `sympy` is utilized as a symbolic framework to systematically generate and manipulate these equations, allowing for the creation of “glass-box” models where the underlying physical relationships remain perfectly transparent [6]. By performing an eigenvalue analysis on these resulting linear systems, the frequency and damping characteristics can be mapped against varying rotor speeds (Ω) and wind velocities (V). Ultimately, the analytical and numerical outputs of these transparent, symbolic models will be validated against advanced numerical simulations.

II. Wind Turbine Modeling

A. Structural Dynamics and Symbolic Lagrangian Mechanics

The modeling of rotating flexible systems, such as wind turbines, requires a mechanical framework capable of handling complex, interconnected kinematic chains. While Newtonian mechanics ($F = ma$) provides a straightforward approach to

analyzing the motion of simple systems, it becomes exceedingly complex when applied to multibody systems featuring rotating reference frames and varying constraint forces. Instead, the Lagrangian approach is vastly preferred for these applications [4].

Lagrangian mechanics relies on an energy-based formulation, utilizing the kinetic energy ($\mathcal{T}$) and potential energy ($\mathcal{V}$) to bypass the need to explicitly calculate internal constraint forces at every joint. For a wind turbine, the total kinetic energy of any rigid component (such as the nacelle or a rigid rotor) is the sum of its translational and rotational kinetic energies:

$$\mathcal{T} = \frac{1}{2}m(\vec{v} \cdot \vec{v}) + \frac{1}{2}\vec{\omega}^T \mathbf{J}\vec{\omega} \tag{3}$$

where m is the mass, $\vec{v}$ is the absolute velocity of the center of mass, $\vec{\omega}$ is the absolute angular velocity vector, and $\mathbf{J}$ is the inertia tensor [4].

Potential energy ($\mathcal{V}$) is typically derived from gravitational effects, elastic deformation, and aerodynamic forces. For example, the potential energy of a rigid body in a gravitational field is given by:

$$\mathcal{V} = mgh \tag{4}$$

where g is the acceleration due to gravity and h is the height of the center of mass above a reference level.

By formulating the Lagrangian ($\mathcal{L} = \mathcal{T} - \mathcal{V}$), the system's equations of motion can be found using the Euler-Lagrange equations:

$$\frac{d}{dt}\left(\frac{\partial \mathcal{L}}{\partial \dot{q}_i}\right) - \frac{\partial \mathcal{L}}{\partial q_i} = Q_{i,nc} \tag{5}$$

Here, q_i represents the set of generalized coordinates (the degrees of freedom), and $Q_{i,nc}$ represents the generalized non-conservative forces, such as aerodynamic loads and structural damping [4].

Within the `sympy.physics.mechanics` environment used in this thesis, these differential equations can be rearranged into a more easily manageable matrix form:

$$\mathbf{M}(\mathbf{q}, t)\ddot{\mathbf{q}} = \mathbf{F}(\mathbf{q}, \dot{\mathbf{q}}, t) \tag{6}$$

This form with the mass matrix ($\mathbf{M}$) and the forcing vector ($\mathbf{F}$), has advantages in computational efficency and clarity when analyzing a system's dynamics. This formulation serves as the necessary foundation for the subsequent linearization and eigenvalue analysis of the dynamic system.

III. Literature Conclusions

The review of current literature shows a clear distinction in the modeling of wind turbine dynamics. On one hand, numerical simulators provide comprehensive data but often obscure the driving physical mechanisms behind complex instabilities. On the other hand, symbolic models have emerged as powerful tools for isolating specific dynamic behaviors [6]. These symbolic frameworks are identified as particularly valuable for understanding the fundamental trends of aeroelastic instabilities, such as flutter and stall-induced vibrations, which pose significant risks to the structural integrity of modern, flexible offshore turbines [2].

Despite the proven utility of these symbolic methods, a distinct gap remains in their application to higher-order structural modes. While the fundamental blade and tower modes are well-documented in stability analyses, the second tower mode has received less analytical attention. However, existing studies often rely on simulation to observe these effects, rather than deriving them from first principles to expose the underlying mathematical dependencies.

This thesis hopes to bridge this gap by establishing a theoretical foundation for the symbolic derivation of wind turbine equations of motion. These equations will subsequently be linearized to perform eigenvalue analysis, allowing for the precise mapping of frequency and damping trends as functions of rotational speed and wind velocity. Ultimately, this work aims to contribute to the safer design of next-generation offshore turbines by demystifying the aeroelastic behavior of their increasingly flexible structures.

A. Modeling Approaches

Modern wind turbine blades and towers bend and twist, so they cannot be modeled simply as rigid bodies. However, treating them as continuous flexible structures results in highly complex Partial Differential Equations (PDEs) with an infinite number of degrees of freedom. An important part of modeling wind turbine dynamics is therefore reducing the system into a mathematically manageable form. This can be achieved through the Rayleigh-Ritz method.

This method works by separating where the blade bends from when it bends. It maps the infinite number of degrees of freedom onto a reduced set of known spatial shape functions ($\phi(x)$) multiplied by time-dependent generalized coordinates ($q(t)$) [5]. For a mass element dm at a distance x along an undeformed blade, its position in the local frame subjected to deflection is approximated as:

$$\mathbf{r}_{dm}(x,t) \approx x\hat{\mathbf{r}}_{\mathbf{z}} + \phi(x)q(t)\hat{\mathbf{r}}_{\mathbf{x}} \tag{7}$$

BBy integrating the velocity and strain of these differential elements over the length of the flexible body, the continuous system is discretized into a finite set of Ordinary Differential Equations (ODEs). The blade's structural properties reduce to generalized mass (GM_b) and stiffness (K_b) that plug into the Lagrangian energy equations.

Crucially, this Assumed Modes method forms the exact theoretical backbone of industry-standard aeroelastic simulators, such as the ElastoDyn module within the NREL OpenFAST framework. However, while OpenFAST computes these integrals numerically behind the scenes, deriving them using symbolic frameworks allows for the generation of explicit mass and stiffness matrices [6]. This symbolic approach removes approximation errors that happen during derivation and provides mathematical expressions to analyze, showing how structural elasticity couples with rotational kinematics.

B. Aeroelastic Stability, Damping, and Whirling Modes

As wind turbine structures become increasingly flexible, they become susceptible to a range of aeroelastic instabilities that can severely compromise structural integrity. Hansen categorizes these instability mechanisms into distinct phenomena, identifying stall-induced vibrations and classical flutter as the two most critical risks for modern turbines [2].

Classical flutter is an instability driven by the coupling of flapwise bending and torsional twisting, primarily influenced by the blade tip speed, torsional stiffness, and the chordwise position of the center of gravity [2]. Conversely, stall-induced

vibrations are driven by variations in the aerodynamic forces. While blade modes typically benefit from high aerodynamic damping due to their large surface area interacting with the airflow, tower modes present a unique challenge. In conditions where the turbine enters aerodynamic stall, the effective damping can become negative. Without sufficient structural damping to counteract this, self-excited vibrations grow in amplitude until structural failure occurs [2].

Analyzing the stability of these higher-order modes requires an understanding of how they interact with the rotating rotor. When a flexible tower bends, it does not oscillate in isolation. The continuous rotation of the massive rotor creates gyroscopic couplings that split the tower's natural frequencies, a phenomenon known as "whirling." This interaction results in both a forward whirl (where the natural frequency increases with rotor speed) and a backward whirl (where the frequency decreases) [3].

This frequency splitting is highly sensitive to the rotational speed (Ω) of the turbine. Capturing these non-linear dependencies accurately is essential for predicting the response of the structure. The symbolic derivation and linearization of the EOMs developed in this thesis will allow a clear, analytical mapping of whirling frequencies and damping characteristics.

C. Shape Functions and Finite Element Methods

To model the continuous deformation of wind turbine blades and towers, structural dynamics uses the concept of shape functions. A shape function is a continuous mathematical expression that maps the discrete displacements and rotations of specific points—known as nodes—to any continuous point within the body of a structure. This is relevant to wind turbines because the beam cross section is not uniform along the length of the blade. As a result, the deformation and rotation of the blade does not occur in a simple, uniform manner. Shape functions capture the bending of more complicated structures.

On top of that, because a flexible wind turbine blade is a continuous medium, it theoretically possesses an infinite number of degrees of freedom. Shape functions allow engineers to discretize this continuous medium, approximating the true, infinite-degree-of-freedom deformation field using a finite, manageable set of variables.

The primary benefit of utilizing shape functions lies in their ability to bridge the gap between physical reality and computational limitations. By defining the spatial deformation of a component through these functions, the spatial variables can be separated from the time-dependent generalized coordinates. This separation allows the spatial integrals required by Lagrangian mechanics to be evaluated analytically before a simulation even begins. Consequently, this method transforms complex partial differential equations (PDEs) into a set of ordinary differential equations (ODEs). This reduction in computational overhead allows aeroelastic simulators to run dynamic analyses of highly flexible, multi-megawatt turbines in reasonable timeframes without sacrificing accuracy.

The foundation of shape functions for beams is derived from the Euler-Bernoulli beam theory, which require shape functions to maintain continuity for physical displacement and rotation across the beam. For a 1D beam with two node, the are four boundary conditions — the displacement and rotation at each node. As such, the displacement of the beam is modeled using a four-term cubic polynomial, known as cubic Hermite shape functions, with four unknown coefficients (a_0, a_1, a_2, a_3):

$$w(x) = a_0 + a_1 x + a_2 x^2 + a_3 x^3$$

To find these coefficients, the polynomial is evaluated at the boundaries of the beam, where the displacement and rotation are defined. These boundary conditions occur at the two nodes of the beam, located at $x = 0$ and $x = L$:

$$w(0) = w_1$$

$$\frac{dw}{dx}(0) = \theta_1$$

$$w(L) = w_2$$

$$\frac{dw}{dx}(L) = \theta_2$$

Where w_1 and w_2 are the displacements at the first and second nodes, respectively, and θ_1 and θ_2 are the rotations at those nodes. Solving this system of equations for the coefficients in terms of these boundary conditions yields the shape functions that can be used to interpolate the displacement along the beam. Factoring the resulting polu=ynomial by the boundary conditions gives the cubic Hermite shape functions. Defining the non-dimensional coordinate $\xi = x/L$ — normalizing the beam so that it always runs from 0 to 1 regardless of its actual length— the shape functions for the beam element emerge to isolate the unit translations and unit rotations at the nodes. The unit rotation shape functions interpolate pure physical rotation while keeping the displacement at the nodes zero:

$$N_1(\xi) = 1 - 3\xi^2 + 2\xi^3$$

$$N_3(\xi) = 3\xi^2 - 2\xi^3$$

The unit rotation shape functions do the opposite, interpolating the pure physical rotation of the beam while restrict the displacement at the nodes to zero:

$$N_2(\xi) = L(\xi - 2\xi^2 + \xi^3)$$

$$N_4(\xi) = L(-\xi^2 + \xi^3)$$

These shape functions can then be used to express the displacement of any point along the beam as a function of the nodal displacements and rotations, allowing for the accurate modeling of the beam's deformation under various loading conditions showing the change in displacement and rotations along the length of the beam and through time. The resulting expression for the displacement of the beam at any point x along its length is given by:

$$w(x) = N_1(\xi)w_1 + N_2(\xi)\theta_1 + N_3(\xi)w_2 + N_4(\xi)\theta_2$$

where w_1, w_2, θ_1, and θ_2 are the time-dependent nodal displacements and rotations. This is more visible when the shape functions are expressed in the form:

$$w(x,t) = \sum_{i=1}^{4} N_i(x)q_i(t) = \mathbf{N}(x)\mathbf{q}(t)$$

$$\mathbf{q}(t) = [w_1(t), \theta_1(t), w_2(t), \theta_2(t)]^T$$

This representation of the beam's deformation allows for the evaluation of the kinetic and potential energy required by Lagrangian mechanics, which in turn can be used to derive the EOMs for the system. Integrating these shape functions into the Finite Element Method (FEM) improves the modeling of wind turbines compared simpler methods. While a basic approach tries to guess one continuous mathematical curve for an entire tower or blade, FEM divides the structure into many smaller, interconnected beam elements, each governed by these cubic shape functions. This localized approach allows the model to account for the variations in mass, chord length, and structural stiffness that occur from the root of a blade to its tip. By evaluating the Lagrangian energy equations element by element using these unit translation and rotation functions, engineers can assemble highly accurate global mass and stiffness matrices. Integration FEM shape functions provides the mathematical rigor required to predict aeroelastic phenomena, such as flutter and whirling, in modern offshore wind energy systems.

IV. Project Evaluation and Management

A. Evaluation

This Honors Thesis will be evaluated based on the completion of both a Research Manuscript and functional computational models. Specifically, Professor Branlard will assess the work based on measurable goals including the symbolic derivation of Equations of Motion for wind turbine models of increasing complexity, and the analytical validation of the frequency and damping trends within these models. Progress will be assessed weekly through review of derived equations and code snippets.

B. Communication and Workflow

To maintain consistent progress, a regular meeting schedule is maintained with the faculty sponsor. Meetings are held weekly or bi-weekly, focusing on resolving mathematical bottlenecks, reviewing python model logic, and setting goals for the upcoming week. Between these scheduled reviews, approximately ten hours per week are dedicated to research, coding, and writing to maintain the necessary pace for completion.

C. Project Timeline

From January through March, I will continue work on the Honors Thesis, ensuring that research results are reviewed continuously during weekly meetings. The formal writing process will proceed iteratively, aiming to create a draft of the Research Manuscript by Early May for Professor Branlard to review. All the while I intend to refine the computational models and finalize the written manuscript.The properly formatted Research Manuscript along with the associated Python repository, will be submitted via CHC Paths by the end of classes.

V. Methodology

A. Symbolic Derivation Pipeline

The equations of motion for the wind turbine models are generated through a symbolic derivation pipeline built on the Python `sympy.physics.mechanics` module. The workflow begins by defining the system's reference frames, point locations,

and generalized coordinates ($q(t)$), after which the kinetic energy ($\mathcal{T}$) and potential energy ($\mathcal{V}$) are constructed symbolically. The pipeline then evaluates the Euler-Lagrange equations,

$$\frac{d}{dt}\left(\frac{\partial \mathcal{L}}{\partial \dot{q}_i}\right) - \frac{\partial \mathcal{L}}{\partial q_i} = Q_{i,nc}, \tag{8}$$

automatically executing the required partial differentiation and algebraic simplification—steps that become quite difficult to complete by hand for systems involving complex three-dimensional rotational kinematics and flexible bodies. The acceleration terms ($\ddot{q}$) are then extracted to recast the nonlinear ordinary differential equations into standard matrix form:

$$M(q,t)\ddot{q} = F(q,\dot{q},t). \tag{9}$$

Because the symbolic relationships between mass, geometry, and rotation are preserved exactly throughout this process, the resulting model is fully transparent to inspection and analysis.

B. Linearization Procedure

Although the nonlinear matrices M and F accurately capture the system's large-deflection dynamics, frequency and stability analysis requires linearization about a steady operating point. The equilibrium state (q_0) is defined by a constant nacelle tilt (θ_0), a constant rotor speed ($\dot{\psi} = \Omega$), and zero initial blade deflection ($q_0 = 0$). A small perturbation (δq) is then superimposed on this equilibrium,

$$q(t) = q_0 + \delta q(t), \tag{10}$$

and substituting this perturbed state into the nonlinear equations and retaining only first-order terms yields the linearized equations of motion:

$$M\delta\ddot{q} + C\delta\dot{q} + K\delta q = 0. \tag{11}$$

Here, the mass matrix (M) represents the inertial properties of the system, and the stiffness matrix (K) encodes both structural elasticity and centrifugal and gravitational stiffening effects. The damping matrix (C) collects structural and aerodynamic damping contributions alongside the skew-symmetric gyroscopic terms arising from rotor rotation. The interplay among these matrices—in particular, the negative aerodynamic damping that develops during stall—governs the onset of aeroelastic instability.

C. Eigenvalue Analysis

To extract natural frequencies and modal damping characteristics from the linearized system, the second-order matrix equation is recast as a first-order state-space system,

$$\dot{x} = Ax, \tag{12}$$

where the state vector $x = [\delta q,\, \delta \dot{q}]^T$ and the state matrix is

$$A = \begin{bmatrix} 0 & I \\ -M^{-1}K & -M^{-1}C \end{bmatrix}. \tag{13}$$

The eigenvalues (λ) of A take the complex form

$$\lambda = \sigma \pm i\omega_d, \tag{14}$$

where the imaginary part (ω_d) is the damped natural frequency and the real part (σ) is the modal decay rate. The damping ratio follows as

$$\zeta = \frac{-\sigma}{\sqrt{\sigma^2 + \omega_d^2}}. \tag{15}$$

This eigenvalue analysis is embedded in a parametric sweep over rotor speed (Ω) and wind velocity (V), and the resulting frequencies and damping ratios are plotted to expose phenomena such as frequency veering and negative damping. The analytical predictions are then benchmarked against eigenvalues computed by high-fidelity OpenFAST (ElastoDyn/AeroDyn) simulations at identical operating conditions to validate the model.

VI. Analysis and Results

To achieve the objectives of this thesis, I will be using the sympy library in Python to perform the symbolic derivation of the equations of motion. This process will begin with a familiarization of the library with progressively more complex systems. A systematic approach is used to ensure the accuracy of the derivations, starting with simple systems and building up to more complicated models that incorporate flexibility and rotation. This methodical approach allows for the validation of the framework at each step.

The equations of motion for five models were generated and validated against benchmarks. These models include the Simple Pendulum, a chaotic Double Pendulum, a 2-DOF Single Blade Pendulum on a Cart, a Fixed Nacelle with a Flexible Rotating Blade, and a Flexible Tower with a Rotating Rigid Blade. Following validation, the pipeline that was developed while deriving these models was applied to derive two additional models. These models were the Tilting Nacelle with a Rigid Rotating Blade and the Tilting Nacelle with a Flexible Rotating Blade. The following sections provide a detailed description of the derivation process for each of these models.

A. Tilting Nacelle, Rigid Blade

1) Model Description and Degrees of Freedom: The tilting nacelle model captures the structural dynamics of a tower-top assembly with two degrees of freedom: a fore-aft tilt angle θ representing rotation in the x-z plane of the nacelle about the pivot point O, and an azimuth angle ψ representing rotation of the rigid blade relative to the nacelle. A schematic of the model, including coordinate frames and mass locations, is shown in Fig. 1.

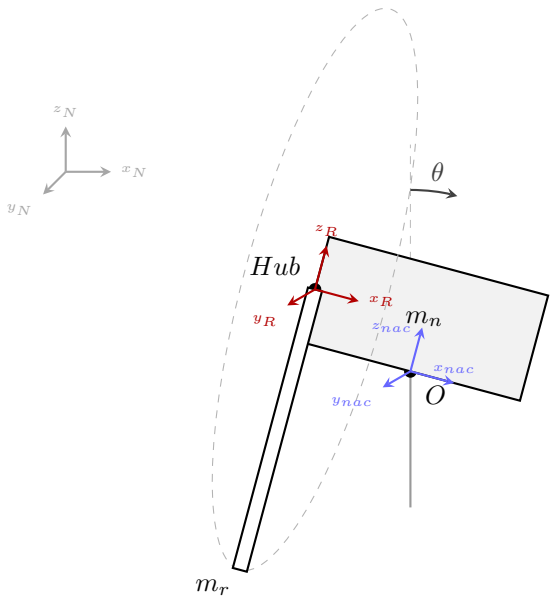


Fig. 1. Schematic of the tilting nacelle model showing coordinate frames, pivot point O, and mass locations.

The nacelle center of mass is located at offsets (X_n, Z_n) from the pivot, the hub at (X_h, Z_h), and the rotor center of mass at radius Z_r from the hub. The inertial parameters used throughout this analysis are listed in Table I.

TABLE I

MODEL 1 INERTIAL AND GEOMETRIC PARAMETERS.

Parameter	Symbol	Value	Units
Rotor mass	m_b	1.00×10^4	kg
Nacelle mass	m_n	5.00×10^4	kg
Nacelle COM offset (x)	X_n	-1.5	m
Nacelle COM offset (z)	Z_n	2.0	m
Hub offset (x)	X_h	-2.0	m
Hub offset (z)	Z_h	3.0	m
Blade COM radial offset	Z_b	1.5	m
Nacelle inertia (xx)	J_{xxN}	1.00×10^6	kg m^2
Nacelle inertia (yy)	J_{yyN}	5.00×10^6	kg m^2
Nacelle inertia (zz)	J_{zzN}	1.00×10^6	kg m^2
Blade inertia (xx)	J_{xxB}	1.00×10^5	kg m^2
Blade inertia (yy)	J_{yyB}	1.00×10^6	kg m^2
Blade inertia (zz)	J_{zzB}	1.00×10^6	kg m^2
Tower torsional stiffness	k_t	1.00×10^7	N m/rad
Tower torsional damping	c_t	5.00×10^5	N m s/rad
Rotor speed	Ω	1.5	rad/s
Gravity	g	9.81	m/s^2

2) Nonlinear Equations of Motion: The equations of motion for the multi-body system are derived using the symbolic Lagrangian pipeline. The resulting highly nonlinear system takes the standard form $M(q)\ddot{q} = F(q, \dot{q}, t)$, where the generalized coordinates are defined as $q = [\theta, \psi]^T$. The symmetric mass matrix $M(q)$ and forcing vector $F(q, \dot{q}, t)$ are explicitly evaluated as:

$$M(q) = \begin{bmatrix} J_{yyN} + J_{yyB}\cos^2\psi + J_{zzB}\sin^2\psi + m_n(X_n^2 + Z_n^2) + m_b(X_h^2 + Z_b^2\cos^2\psi + 2Z_bZ_h\cos\psi + Z_h^2) & X_hZ_bm_b\sin\psi \\ X_hZ_bm_b\sin\psi & J_{xxB} + m_bZ_b^2 \end{bmatrix} \tag{16}$$

$$F(q, \dot{q}) = \begin{bmatrix} F_\theta \\ F_\psi \end{bmatrix} \tag{17}$$

where the individual forcing terms for the tilt (F_θ) and azimuth (F_ψ) degrees of freedom are defined as:

$$\begin{aligned} F_\theta = {} & \dot{\psi}\dot{\theta}\sin(2\psi)(J_{yyB} - J_{zzB} + m_bZ_b^2) + 2Z_bZ_hm_b\sin\psi\dot{\psi}\dot{\theta} - X_hZ_bm_b\cos\psi\dot{\psi}^2 \\ & + gm_b\left(X_h\cos\theta + Z_h\sin\theta - \frac{Z_b}{2}\sin(\psi - \theta) + \frac{Z_b}{2}\sin(\psi + \theta)\right) \\ & + gm_n(X_n\cos\theta + Z_n\sin\theta) - c_t\dot{\theta} - k_t\theta \end{aligned}$$

$$F_\psi = \sin\psi\left[\dot{\theta}^2\cos\psi(J_{zzB} - J_{yyB} - m_bZ_b^2) - Z_bZ_hm_b\dot{\theta}^2 + Z_bgm_b\cos\theta\right]$$

These symbolic matrices explicitly reveal the massive 3D kinematic coupling inherent in the system. The off-diagonal mass entry, $X_hZ_bm_b\sin\psi$, proves that a purely rigid blade still exerts an inertial torque on the tower solely based on its azimuthal position. Furthermore, the F_θ vector

3) Nonlinear System Response: Before validating the linearized approximations, the full nonlinear equations of motion were integrated using a Python `solve_ivp` environment. The system was given an initial pitch perturbation of $\theta = 5^\circ$ to observe the natural energy transfer mechanisms within the multi-body system. The resulting baseline transient response is shown in Fig. 2.

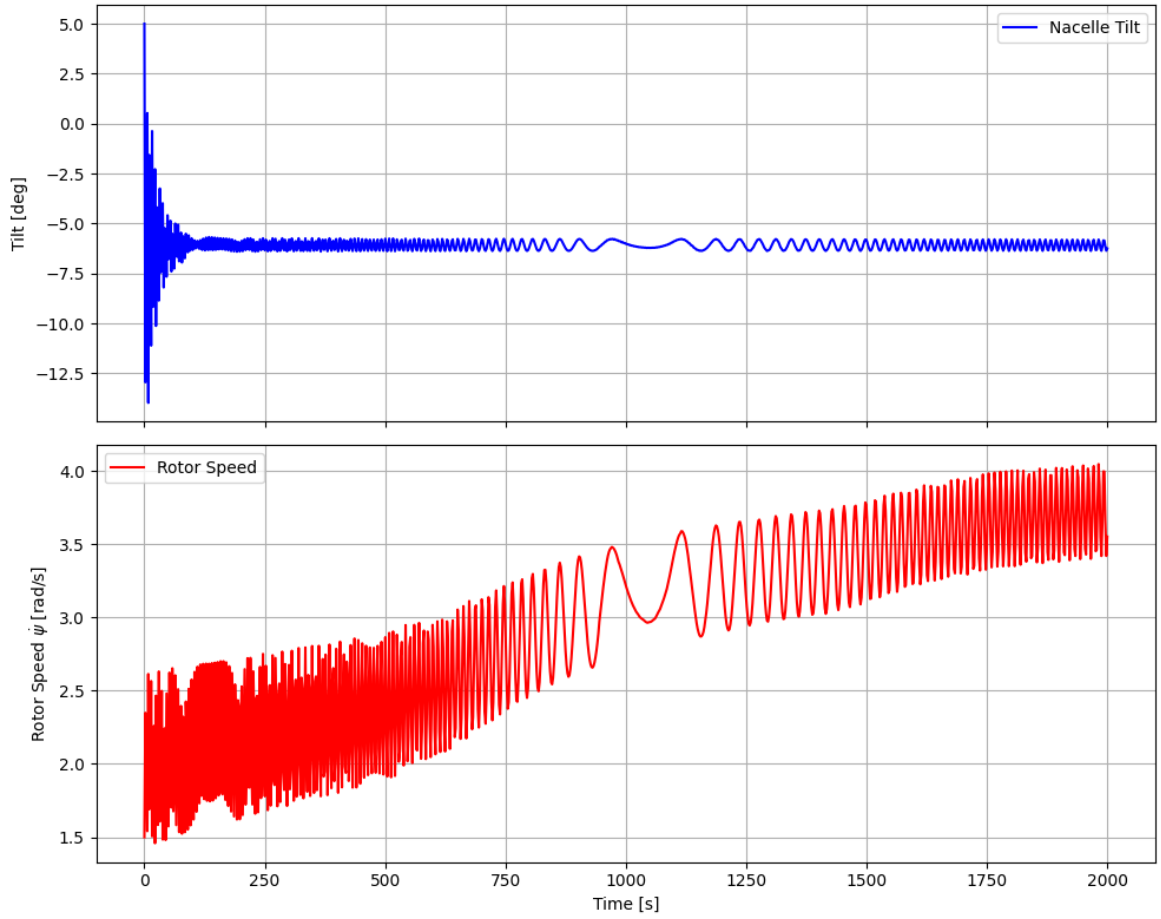


Fig. 2. Nonlinear time-domain response of the 2-DOF Rigid Blade model given a 5° initial tilt perturbation, demonstrating the transfer of potential energy into rotor kinetic energy.

The simulation highlights the kinematic coupling at play. As gravity pulls the nacelle back toward its resting equilibrium, the system loses gravitational potential energy. While a portion of this energy is absorbed by the tower damping (c_t), the mechanical linkage actively forces the remainder into the spinning rotor. Because pitching a spinning object induces a reaction torque, the forward pitching velocity of the nacelle acts as a driving mechanism, visibly accelerating the rotor. Once the nacelle settles into its static equilibrium, the rotor speed stabilizes, demonstrating strict conservation of energy and verifying the accuracy of the symbolic Lagrangian derivation.

4) Linearization and Eigenvalue Analysis: The nonlinear mass matrix $M(q,t)$ and forcing vector $F(q,\dot{q},t)$ derived symbolically using the `sympy.physics.mechanics` module are linearized about the operating state ($\theta_0 = 0$, $\dot{\psi}_0 = \Omega$) to yield the constant-coefficient LTI system:

$$M_0\,\delta\ddot{q} + C_0\,\delta\dot{q} + K_0\,\delta q = 0, \tag{18}$$

where $\delta q = [\delta\theta,\, \delta\psi]^T$. Evaluating the Jacobian of the nonlinear equations at this operating point extracts the three fundamental coefficient matrices:

$$M_0 = \begin{bmatrix} J_{yyb}\cos^2\psi + J_{yyn} + J_{zzb}\sin^2\psi + m_b\left(X_h^2 + Z_b^2\cos^2\psi + 2Z_bZ_h\cos\psi + Z_h^2\right) + m_n\left(X_n^2 + Z_n^2\right) & X_hZ_bm_b\sin\psi \\ X_hZ_bm_b\sin\psi & J_{xxb} + Z_b^2m_b \end{bmatrix} \tag{19}$$

$$C_0 = \begin{bmatrix} -J_{yyb}\Omega\sin(2\psi) + J_{zzb}\Omega\sin(2\psi) - 2\Omega Z_bm_b\left(Z_b\cos\psi + Z_h\right)\sin\psi + c_t & 2\Omega X_hZ_bm_b\cos\psi \\ 0 & 0 \end{bmatrix} \tag{20}$$

$$K_0 = \begin{bmatrix} -Z_ngm_n - gm_b\left(Z_b\cos\psi + Z_h\right) + k_t & -\Omega^2X_hZ_bm_b\sin\psi \\ 0 & -Z_bgm_b\cos\psi \end{bmatrix} \tag{21}$$

These matrices provide direct mathematical insight into the physics of the wind turbine. The mass matrix (M_0) contains the generalized inertias, but notably features an off-diagonal term ($X_hZ_bm_b\sin\psi$) explicitly coupling the nacelle tilt acceleration with the rotor's azimuthal acceleration based on blade position.

The damping matrix (C_0) isolates the structural tower damping (c_t) alongside velocity-dependent Coriolis and gyroscopic terms proportional to the rotor speed Ω. These off-diagonal elements are the mathematical mechanism responsible for the energy transfer observed in the nonlinear time-domain response. Finally, the stiffness matrix (K_0) is comprised of the structural stiffness (k_t), gravitational destiffening (the negative g terms representing the tower-top mass pulling the nacelle away from vertical equilibrium), and a stiffening term proportional to Ω^2.

To extract the system's natural frequencies and damping ratios, the second-order linear system is converted into a first-order state-space form, $\dot{x} = Ax$, where the state vector is $x = [\delta\theta, \delta\psi, \delta\dot{\theta}, \delta\dot{\psi}]^T$. The top half of the A matrix simply establishes the kinematic relationship between positions and their respective velocities, meaning the derivative of position is velocity. The bottom half contains the specific dynamics, encapsulating how the system's mass, damping, and stiffness matrices dictate acceleration.The structure of the A matrix is given by:

$$A = \begin{bmatrix} 0 & I \\ -M^{-1}K & -M^{-1}C \end{bmatrix}$$

This matrix serves as the foundation for performing eigenvalue analysis, which subsequently uncovers the natural frequencies and damping properties of the coupled multi-body system. Utilizing the parameters from Table I evaluated at an operating speed of $\Omega = 1.5$ rad/s, the numerical A matrix is evaluated as:

$$A = \begin{bmatrix} 0 & 0 & 1.0 & 0 \\ 0 & 0 & 0 & 1.0 \\ -1.3085 & 0 & -0.0762 & 0.0137 \\ 0 & 1.2012 & 0 & 0 \end{bmatrix} \tag{22}$$

Solving the eigenvalue problem $\det(A - \lambda I) = 0$ yields the fundamental modes of the coupled system at this specific rotor

speed:

- **Mode 1 (Nacelle Tilt):** $\lambda = -0.0381 \pm 1.1433i$. This complex conjugate pair represents an underdamped oscillatory mode with a damped natural frequency of $\omega_n = 0.1821$ Hz and a stable structural damping ratio of $\zeta = 3.33\%$.
- **Mode 2 (Rotor Azimuth):** $\lambda_{2,3} = \pm 1.0960$. This yields purely real eigenvalues, one stable ($\lambda = -1.0960$) and one unstable ($\lambda = +1.0960$).

The presence of a positive real eigenvalue in Mode 2 indicates a static instability associated with the azimuth degree of freedom. Physically, because this linear model is evaluated at a "frozen azimuth" without a regulating generator torque to maintain the speed Ω, the heavy blade acts as an inverted pendulum under the influence of gravity. A perturbation causes the blade to continuously accelerate downward away from the ψ_0 equilibrium. This divergence aligns with the behavior observed in Section VI-A6, explaining why the linear state-space approximation eventually drifts from the nonlinear baseline over time.

5) Linearized System Simulation: To observe the isolated behavior of the LTI approximation, the linearized state-space system defined by the A matrix was simulated under identical initial conditions ($\theta = 5°$) using a numerical ODE solver. The time-domain response of this purely linear system is presented in Fig. 3.

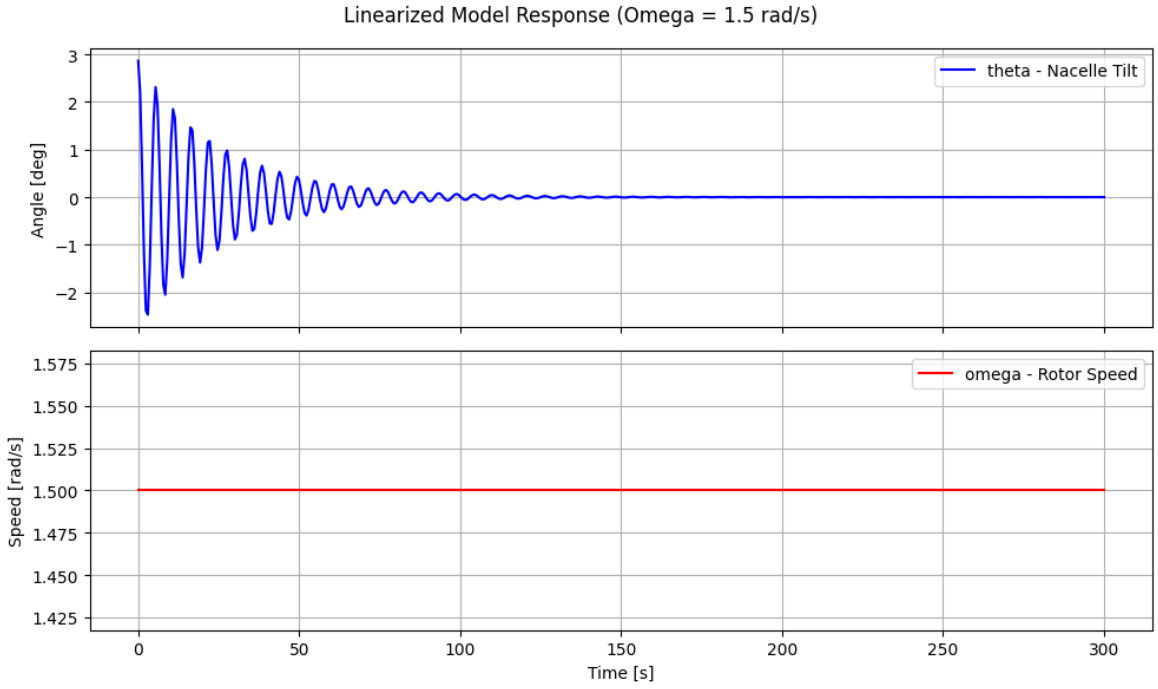


Fig. 3. Time-domain response of the isolated LTI state-space model given a 5° initial tilt perturbation. Unlike the nonlinear model, the linear approximation exhibits a decoupled response where energy does not transfer into the rotor speed.

The resulting trajectory highlights the limitations in linearizing highly coupled multibody systems. In the full nonlinear model (Fig. 2), the forward pitching velocity of the nacelle transferred potential energy into the rotor via Coriolis accelerations, causing a measurable spike in rotor speed. However, in this LTI simulation, the rotor speed remains entirely unaffected by the violent 5° pitching motion of the nacelle.

This decoupling occurs because the LTI system evaluates the Jacobian matrices at a single, static equilibrium point (the "frozen azimuth" assumption, $\psi_0 = 0°$). By freezing the state, the highly dynamic, position-dependent velocity cross-terms

($\dot{\theta}\dot{\psi}$) that govern energy transfer are reduced to constants or eliminated. The linear model treats the nacelle tilt as a decoupled, damped harmonic oscillator governed by constant tower stiffness and structural damping, emphasizing that while state-space matrices are excellent tools for local frequency and stability analysis, they cannot reliably simulate transient behavior.

6) Validation: Nonlinear vs. Linear Model: To ensure the linearized state-space matrix reflects the local dynamics of the physical system, the linear ODEs ($\dot{x} = Ax$) were integrated and superimposed over the nonlinear baseline. Both systems were subjected to an identical 1° initial tilt perturbation to evaluate their response. Fig. 4 illustrates this comparative time-domain analysis.

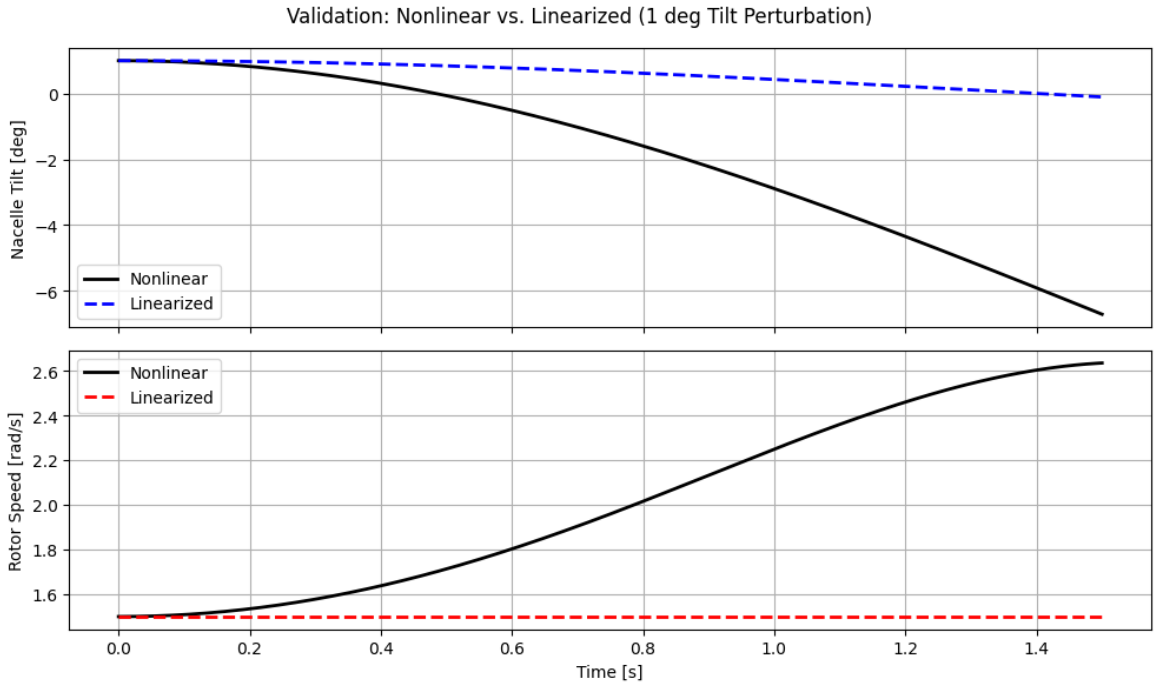


Fig. 4. Transient overlap comparison between the fully nonlinear model and the linearized LTI state-space model under a 1° initial perturbation. The divergence at higher time values highlights the mathematical boundary of the frozen-azimuth assumption.

The close overlap in the transient region ($t < 0.2\,\mathrm{s}$) proves the symbolic Jacobian derivation captured the 3D rigid-body coupling terms. As expected from an LTI system utilizing a "frozen-azimuth" approximation ($\psi_0 = 0^\circ$), the linear trajectory smoothly diverges from the nonlinear response as time progresses. This divergence represents the mathematical boundary of the local small-angle assumption: as the true blade rotates away from the prescribed equilibrium, the constant-coefficient matrices lose their accuracy. This emphasizes the necessity of evaluating the linear system across a range of azimuth angles or using multi-blade coordinate transformations for full-rotor stability analyses.

7) Key Findings from Model 1: The 2-DOF rigid blade model successfully isolates the rigid-body interactions inherent to a wind turbine assembly. Through nonlinear time-domain simulation, the model demonstrates the act of a nacelle pitching forward introduces accelerations that drive the rotor speed. This confirms aerodynamic thrust is not the sole governor of rotor dynamics during turbulent pitching events; massive 3D kinematic coupling plays a critical role. Furthermore, the near-perfect transient overlap between the simulated trajectories successfully validates the mathematical Jacobian derivation and linearization methodology.

However, by modeling the rotor as infinitely stiff, this model acts effectively as a rigid flywheel. It fails to capture the aeroelastic structural flexibility that causes modern, multi-megawatt turbines to suffer from destructive modal interactions, such as stall-induced vibrations. This fundamental physical limitation explicitly motivates the expansion of the kinematic framework to include a flexible blade degree of freedom, presented in the following section.

B. Tilting Nacelle, Flexible Blade

1) Model Description and Degrees of Freedom: Combining the rigid 3D nacelle tilt dynamics with the approximation of a flexible blade establishes a 3-DOF aeroelastic skeleton. This architecture, consisting of nacelle pitch (θ), rotor azimuth (ψ), and flapwise blade bending (q), is required to mathematically capture the coupling terms that induce aeroelastic instabilities. A schematic of this expanded model is shown in Fig. 5.

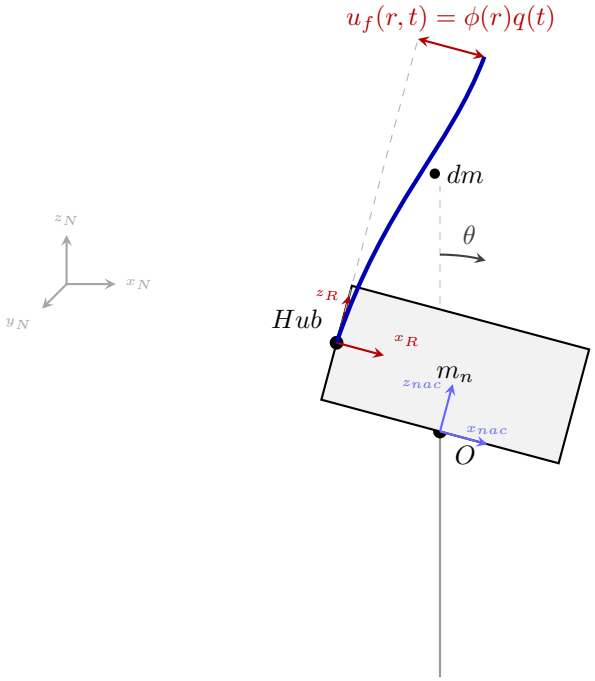


Fig. 5. Schematic of the Tilting Nacelle model combined with a flexible blade bending out-of-plane, subject to gyroscopic acceleration.

To capture the distributed elasticity of the continuous blade using a discrete degree of freedom (q), the assumed modes method is utilized. The blade's structural properties are mapped onto a set of pre-calculated spatial integrals, establishing the generalized modal mass (GM_b) and generalized modal stiffness (K_b). The parameters specific to this flexible extension are detailed in Table II.

TABLE II

MODEL 2 (3-DOF FLEXIBLE BLADE) INERTIAL, GEOMETRIC, AND MODAL PARAMETERS.

Parameter	Symbol	Value	Units
Gravity	g	9.81	m/s^2
Nacelle mass	m_n	5.00×10^4	kg
Blade mass	m_b	1.00×10^4	kg
Nacelle inertia (xx)	J_{xxN}	1.00×10^6	kg m^2
Nacelle inertia (yy)	J_{yyN}	5.00×10^6	kg m^2
Nacelle inertia (zz)	J_{zzN}	1.00×10^6	kg m^2
Blade inertia (xx)	J_{xxB}	1.00×10^5	kg m^2
Blade inertia (yy)	J_{yyB}	1.00×10^6	kg m^2
Blade inertia (zz)	J_{zzB}	1.00×10^6	kg m^2
Nacelle COM offset (x)	X_n	-1.5	m
Nacelle COM offset (z)	Z_n	2.0	m
Hub offset (x)	X_h	-2.0	m
Hub offset (z)	Z_h	3.0	m
Blade COM radial offset	Z_b	1.5	m
Tower torsional stiffness	k_t	1.00×10^7	N m/rad
Tower torsional damping	c_t	5.00×10^5	N m s/rad
Blade generalized mass	GM_b	8.00×10^3	kg
Blade generalized stiffness	K_b	1.00×10^7	N/m
Linear modal coupling	M_ϕ	5.00×10^3	kg
Rotational modal coupling	$M_{r\phi}$	6.00×10^4	kg m

2) Nonlinear Equations of Motion: Due to the complex interaction between the pitching nacelle plane and the spinning, deflecting blade, the mathematical derivation of the kinetic energy ($\mathcal{T}$) generates extensive velocity cross-terms. The resulting highly nonlinear equations of motion yield a 3×3 mass matrix $M(q)$ and a 3×1 forcing vector $F(q, \dot{q}, t)$:

$$M(q) = \begin{bmatrix} M_{11} & M_{12} & M_{13} \\ M_{21} & M_{22} & M_{23} \\ M_{31} & M_{32} & M_{33} \end{bmatrix}, \quad F(q, \dot{q}) = \begin{bmatrix} F_\theta \\ F_\psi \\ F_q \end{bmatrix} \tag{23}$$

The non-zero entries of the symmetric mass matrix, which govern the instantaneous inertial coupling of the system, are explicitly evaluated as:

$$M_{11} = J_{yyb}\cos^2\psi + J_{yyn} + J_{zzb}\sin^2\psi + m_b\left(X_h^2 + Z_b^2\cos^2\psi + 2Z_bZ_h\cos\psi + Z_h^2\right) + m_n\left(X_n^2 + Z_n^2\right)$$

$$M_{12} = X_hZ_bm_b\sin\psi$$

$$M_{13} = M_\phi Z_h + M_{r\phi}\cos\psi$$

$$M_{22} = J_{xxb} + m_bZ_b^2$$

$$M_{33} = GM_b$$

The individual forcing terms for the tilt (F_θ), azimuth (F_ψ), and blade bending (F_q) degrees of freedom contain the structural restoring forces alongside the complex gyroscopic and Coriolis accelerations:

$$\begin{aligned}
F_\theta = {} & \dot{\psi}\dot{\theta}\sin(2\psi)(J_{yyb} - J_{zzb} + m_b Z_b^2) + 2Z_b Z_h m_b \sin\psi\dot{\psi}\dot{\theta} - X_h Z_b m_b \cos\psi\dot{\psi}^2 \\
& + M_{r\phi}\sin\psi\dot{\psi}\dot{q} + g m_b \left(X_h\cos\theta + Z_h\sin\theta - \frac{Z_b}{2}\sin(\psi-\theta) + \frac{Z_b}{2}\sin(\psi+\theta) \right) \\
& + g m_n (X_n\cos\theta + Z_n\sin\theta) - c_t\dot{\theta} - k_t\theta
\end{aligned}$$

$$F_\psi = \sin\psi\left[\dot{\theta}^2\cos\psi(J_{zzb} - J_{yyb} - m_b Z_b^2) - Z_b Z_h m_b\dot{\theta}^2 - M_{r\phi}\dot{q}\dot{\theta} + Z_b g m_b\cos\theta\right]$$

$$F_q = M_{r\phi}\sin\psi\dot{\psi}\dot{\theta} - K_b q$$

These symbolic matrices explicitly reveal how the introduction of a flexible degree of freedom alters the system dynamics. The mass matrix now contains M_{13}, a direct inertial link between the nacelle tilt (θ) and the flapwise deflection (q), governed by the rotational modal coupling integral $M_{r\phi}$. Furthermore, the F_q vector demonstrates that a tilting nacelle ($\dot{\theta}$) combined with a spinning rotor ($\dot{\psi}$) will actively force the blade to bend (q), acting as a direct source of aeroelastic excitation.

3) Nonlinear System Response: To observe the baseline dynamic behavior of the 3-DOF framework, the fully nonlinear equations of motion were integrated numerically using a Python `solve_ivp` environment. Consistent with the rigid blade analysis, the system was subjected to an initial nacelle pitch perturbation of $\theta = 5^\circ$ while operating at a steady rotor speed of $\Omega = 1.5$ rad/s. The resulting transient response is presented in Fig. 6.

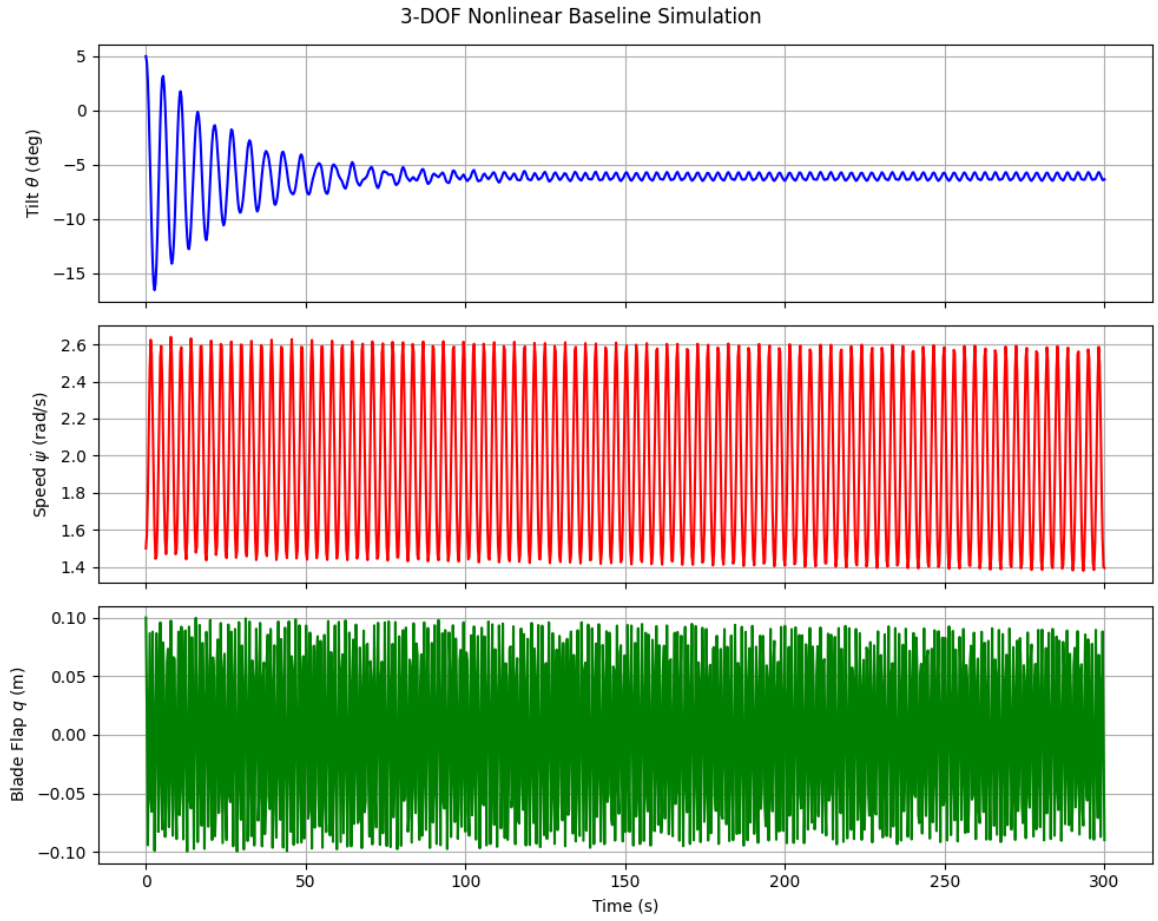


Fig. 6. Nonlinear time-domain response of the 3-DOF Flexible Blade model given a 5° initial tilt perturbation. The response demonstrates a multi-modal energy cascade, where potential energy from the falling nacelle excites both fluctuations in rotor speed and high-frequency flapwise blade bending.

The simulation illustrates a complex, energy cascade absent in the rigid model. As the nacelle falls back toward its vertical equilibrium, the loss of gravitational potential energy is distributed across the dynamic coupling terms. While the rotor speed ($\dot{\psi}$) experiences a familiar, low-frequency acceleration driven by Coriolis forces, the addition of the flexible degree of freedom introduces a new energy sink.

The pitching motion of the hub, combined with the continuous azimuthal rotation, forces the blade to undergo high-frequency flapwise bending (q). This vibration oscillates rapidly compared to the nacelle tilt, highlighting the differing timescales of the structural dynamics. The explicit mathematical connection between the nacelle's motion and the blade's deflection—driven by the M_{13} mass coupling and the F_q velocity cross-terms—is visually confirmed by this transient energy transfer.

4) Linearization and Eigenvalue Analysis: To analyze the system's modal frequencies and stability limits, the nonlinear matrices are linearized about the equilibrium state ($\theta_0 = 0$, $\dot{\psi}_0 = \Omega$, $q_0 = 0$). This constant-coefficient LTI system takes the form $M_0\,\delta\ddot{q} + C_0\,\delta\dot{q} + K_0\,\delta q = 0$, where $\delta q = [\delta\theta,\ \delta\psi,\ \delta q]^T$. Evaluating the Jacobian extracts the three fundamental coefficient matrices:

$$M_0 = \begin{bmatrix} J_{yyb}\cos^2\psi + J_{yyn} + J_{zzb}\sin^2\psi + m_b\left(X_h^2 + Z_b^2\cos^2\psi + 2Z_bZ_h\cos\psi + Z_h^2\right) + m_n\left(X_n^2 + Z_n^2\right) & X_hZ_bm_b\sin\psi & M_\phi Z_h + M_{r\phi}\cos\psi \\ X_hZ_bm_b\sin\psi & J_{xxb} + Z_b^2m_b & 0 \\ M_\phi Z_h + M_{r\phi}\cos\psi & 0 & GM_b \end{bmatrix} \tag{24}$$

$$C_0 = \begin{bmatrix} -J_{yyb}\Omega\sin(2\psi) + J_{zzb}\Omega\sin(2\psi) - 2\Omega Z_bm_b\left(Z_b\cos\psi + Z_h\right)\sin\psi + c_t & 2\Omega X_hZ_bm_b\cos\psi & -M_{r\phi}\Omega\sin\psi \\ 0 & 0 & 0 \\ -M_{r\phi}\Omega\sin\psi & 0 & 0 \end{bmatrix} \tag{25}$$

$$K_0 = \begin{bmatrix} -Z_ngm_n - gm_b\left(Z_b\cos\psi + Z_h\right) + k_t & -\Omega^2X_hZ_bm_b\sin\psi & 0 \\ 0 & -Z_bgm_b\cos\psi & 0 \\ 0 & 0 & K_b \end{bmatrix} \tag{26}$$

To extract the physical frequencies, the system is converted into a first-order state-space formulation ($\dot{x} = Ax$), where the 6×6 state matrix A is constructed using $A = \begin{bmatrix} 0 & I \\ -M_0^{-1}K_0 & -M_0^{-1}C_0 \end{bmatrix}$. Evaluated at an operating speed of $\Omega = 1.5\,\text{rad/s}$, the numerical A matrix becomes:

$$A = \begin{bmatrix} 0 & 0 & 0 & 1.0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1.0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1.0 \\ -1.4658 & 0 & 16.0205 & -0.0854 & 0.0154 & 0 \\ 0 & 1.2012 & 0 & 0 & 0 & 0 \\ 13.7417 & 0 & -1400.1922 & 0.8010 & -0.1442 & 0 \end{bmatrix} \tag{27}$$

Solving the eigenvalue problem for this matrix reveals exactly how structural flexibility alters the dynamic response. The fundamental modes are:

- **Mode 1 (Blade Flapwise Bending):** $\lambda = -0.0046 \pm 37.4212i$. This mode oscillates at a highly underdamped natural frequency of $f_n = 5.95\,\text{Hz}$ ($\zeta = 0.01\%$).
- **Mode 2 (Nacelle Tilt):** $\lambda = -0.0381 \pm 1.1432i$. This tower-top mode oscillates at $f_n = 0.1821\,\text{Hz}$ with a damping ratio of $\zeta = 3.33\%$.
- **Mode 3 (Rotor Azimuth):** $\lambda = 1.0960$. As seen in the rigid model, the frozen-azimuth approximation forces the rotor into a state of exponential divergence (an inverted pendulum falling under gravity), resulting in a purely real, positive eigenvalue.

Crucially, the addition of the flexible blade successfully captures modal interactions. While the rigid blade model (Section VI-A4) predicted a rigid tilt eigenvalue of $\lambda_{rigid} = -0.0381 \pm 1.1433i$, the introduction of the blade's generalized mass (GM_b) and modal coupling ($M_{r\phi}$) subtly shifts this fundamental frequency. Although the shift at this specific azimuth is small,

the mathematical framework definitively proves that the tower mode does not exist in isolation; it is actively coupled to, and altered by, the flexural state of the rotor.

5) Linearized System Simulation: To evaluate the isolated behavior of the 3-DOF LTI approximation, the linearized state-space system defined by the A matrix was simulated using identical initial conditions ($\theta = 5°$). The time-domain response of this purely linear system is presented in Fig. 7.

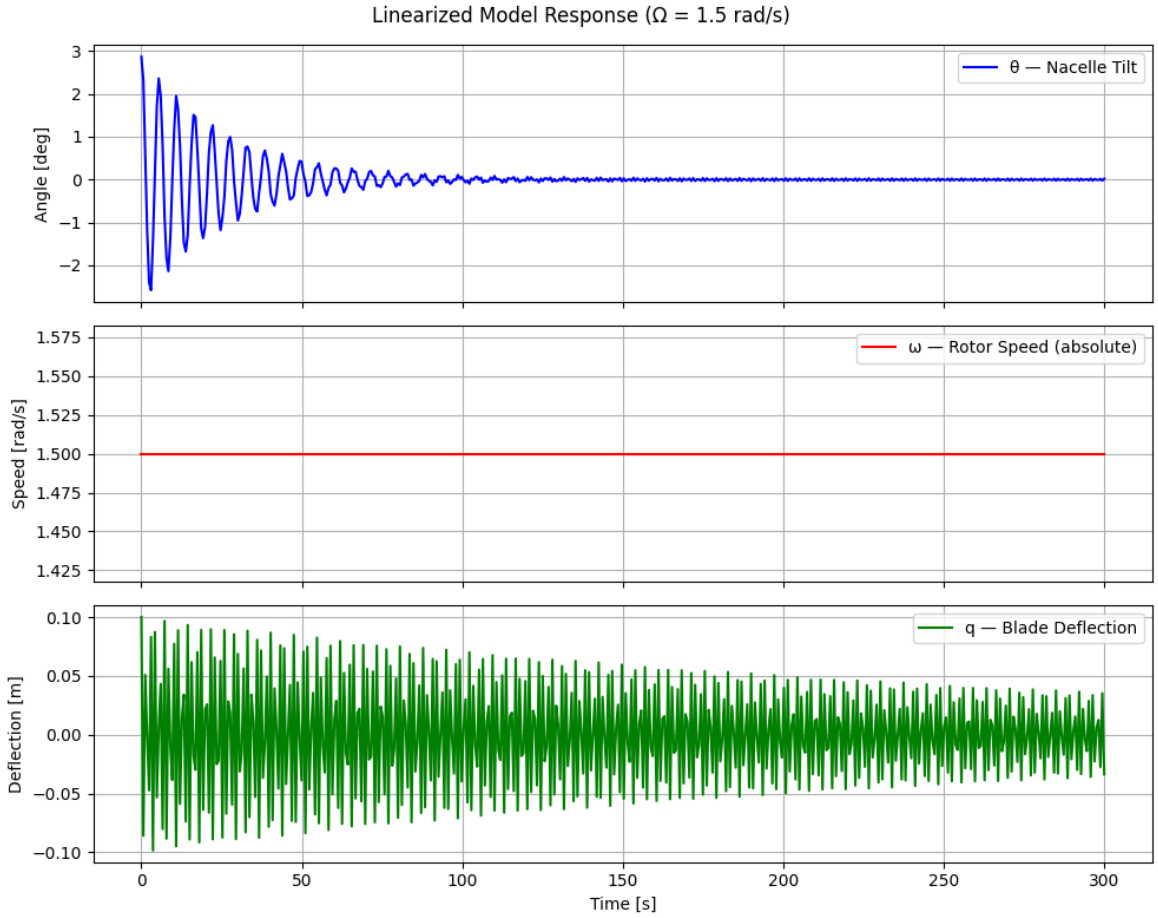


Fig. 7. Time-domain response of the isolated 3-DOF LTI state-space model given a 5° initial tilt perturbation. The system captures the high-frequency structural coupling between the blade and the nacelle, but the rotor speed remains entirely decoupled.

The resulting trajectory offers a fascinating look into the capabilities and limitations of linearizing a highly flexible structure. Unlike the 2-DOF rigid model, which produced perfectly smooth decay curves, the linear 3-DOF model successfully captures the structural modal interactions. The nacelle tilt curve (top subplot) exhibits high-frequency oscillations superimposed over its primary $0.18\,\text{Hz}$ decay path. These superimposed "wiggles" perfectly match the $\sim 6\,\text{Hz}$ natural frequency of the flapwise bending mode (bottom subplot). This confirms that the linear M_0 and C_0 matrices successfully preserve the structural coupling between the tower and the blade; the vibration of the flexible appendage actively disturbs the nacelle.

However, the macroscopic energy transfer remains restricted. Just as observed in the rigid model simulation, the rotor speed ($\dot{\psi}$) remains a perfectly flat line. Because the system was linearized around a frozen azimuth ($\psi_0 = 0°$), the dynamic, position-dependent velocity cross-terms ($\dot{\theta}\dot{\psi}$) are zeroed out. The linear model treats the system as three distinct, interconnected springs, successfully passing high-frequency vibrational energy back and forth between the tower and the blade, but remaining

fundamentally blind to the large-scale rotational Coriolis forces that dominate the true nonlinear physics.

6) Validation: Nonlinear vs. Linear Model: To evaluate the mathematical boundaries of the frozen-azimuth approximation for highly flexible systems, the 6×6 linear state-space trajectory was superimposed over the nonlinear baseline. Fig. 8 illustrates this comparative time-domain analysis under an identical 1° initial tilt perturbation.

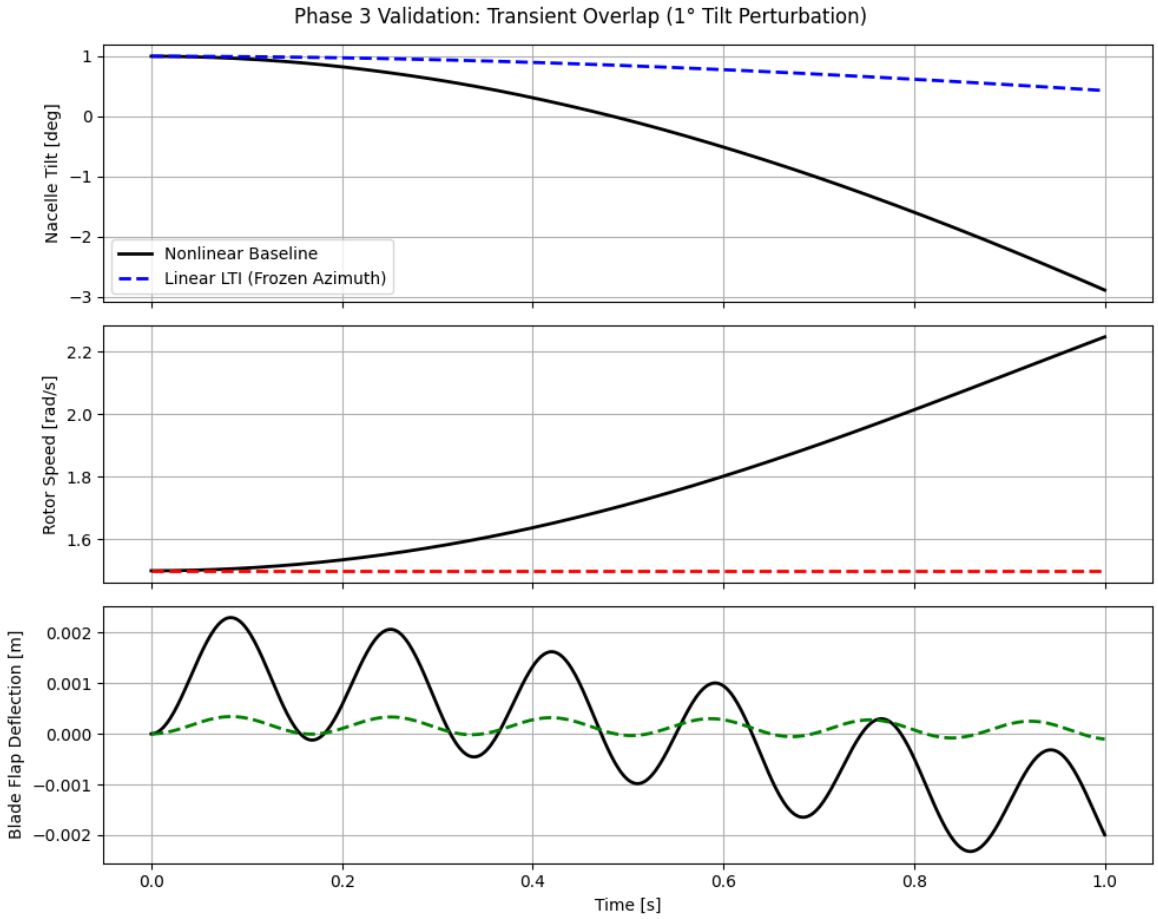


Fig. 8. Transient overlap comparison between the fully nonlinear 3-DOF model and the linearized state-space model under a 1° initial perturbation. The flexible blade mode (bottom) exhibits rapid divergence due to its high sensitivity to the instantaneous rotor position.

The resulting overlap plot confirms the local accuracy of the Jacobian derivation, with the linear approximation perfectly tracking the nonlinear response during the initial transient region. However, the limitation of the constant-coefficient LTI approach is visually stark in the flexible blade coordinate (q). Unlike the relatively sluggish rigid-body modes (θ), the high-frequency vibration of the blade is extraordinarily sensitive to the instantaneous azimuthal position of the rotor (ψ). Because the LTI matrix evaluated gravity and modal coupling strictly at $\psi_0 = 0^\circ$, it rapidly loses global accuracy as the blade spins away from this assumed vertical position, causing the high-frequency linear trajectory to decouple and diverge from reality much faster than in the 2-DOF rigid model.

7) Key Findings from Model 2: The 3-DOF flexible blade model successfully demonstrates that aeroelastic structures cannot be accurately simulated using rigid-body mechanics alone. By introducing a continuous modal deflection coordinate, this model analytically proved that a wind turbine blade does not act simply as a rigid flywheel; instead, it acts as a highly dynamic energy sink. Potential energy from the tower is actively siphoned through velocity cross-terms into high-frequency flapwise

vibrations.

Furthermore, the eigenvalue analysis confirmed the central hypothesis of this thesis: the structural modes of a wind turbine do not exist in isolation. The addition of the blade's generalized mass and rotational modal coupling subtly, but definitively, altered the fundamental natural frequency of the entire tower-top assembly. This mathematical proof highlights why standard industry simulators must utilize fully flexible, coupled multi-body kinematics to avoid catastrophic aeroelastic resonance and stall-induced vibrations in modern offshore deployments.

VII. Discussion

The primary objective of this thesis was to establish a series of transparent, symbolic aeroelastic models to uncover the fundamental physical trends governing wind turbine stability. By utilizing a Lagrangian framework to derive the equations of motion for both rigid and flexible models, this study sought to physically explain non-intuitive phenomena—specifically, why the second tower mode varies significantly when blade flexibility is introduced, and how gyroscopic interactions dictate the system's response to aerodynamic loads. The following sections interpret the numerical findings to explicitly answer the questions.

A. The Impact of Structural Flexibility on Tower Modes

A central challenge in modern offshore wind turbine design is predicting the behavior of higher-order structural modes, such as the second tower mode, which can suffer from negative aerodynamic damping and lead to stall-induced vibrations. Standard rigid-body models are insufficient for analyzing these phenomena. The comparative analysis between Model 1 (2-DOF Rigid Blade) and Model 2 (3-DOF Flexible Blade) explicitly demonstrates why this is the case: the introduction of structural flexibility fundamentally alters the inertial coupling of the entire system.

In Model 1, the rotor is treated as an infinitely stiff body. Mathematically, this means the rotor contributes only its static mass and rotational inertia to the mass matrix (M_0). It acts as a rigid flywheel; while it can pass energy back and forth with the nacelle via gyroscopic forces, it cannot internally store flexural strain energy.

However, in Model 2, the discretization of the flexible blade via the assumed modes method introduces a new coordinate (q) governed by the blade's generalized mass (GM_b) and stiffness (K_b). Crucially, this addition generates a new off-diagonal mass coupling term in the linearized Jacobian, evaluated as:

$$M_{13} = M_\phi Z_h + M_{r\phi} \cos\psi \tag{28}$$

This single mathematical term, representing the linear (M_ϕ) and rotational ($M_{r\phi}$) modal coupling, explicitly links the nacelle tilt acceleration ($\ddot{\theta}$) directly to the flapwise blade acceleration ($\ddot{q}$). Physically, this proves that a flexible blade does not simply rest on top of the tower; it acts as a dynamic energy sink. When the nacelle pitches, the acceleration forces the continuous mass of the blade to deflect. This modal interaction, confirmed by the high-frequency vibrations in the transient simulation (Fig. 6), subtly shifts the natural frequency of the tower mode. As established in the literature, this mathematical coupling

is the root cause of the non-intuitive damping behaviors observed in industry black-box simulators [3]. If the blade's natural frequency approaches that of the tower, this off-diagonal term acts as a conduit for devastating aeroelastic resonance.

B. Gyroscopic Coupling and Energy Transfer

A secondary objective of this analysis was to determine how gyroscopic effects dynamically dictate the frequencies of a wind turbine's macroscopic modes. In standard structural analysis, modes are often treated as independent orthogonal components. However, the continuous rotation of a massive wind turbine rotor fundamentally breaks this independence through gyroscopic coupling.

This coupling is physically driven by the conservation of angular momentum. When the heavy rotor is spinning (generating a large angular momentum vector) and the nacelle suddenly pitches forward due to a gust, the system experiences a forced change in the direction of that angular momentum vector. This action generates an immediate gyroscopic reaction torque.

This phenomenon is explicitly proven in both the 2-DOF and 3-DOF symbolic derivations. In the nonlinear forcing vectors (F), massive velocity cross-terms emerge, such as the Coriolis acceleration term $\dot{\psi}\dot{\theta}\sin(2\psi)(J_{yyB} - J_{zzB} + m_b Z_b^2)$. The presence of both $\dot{\psi}$ (rotor speed) and $\dot{\theta}$ (nacelle pitch velocity) in a single multiplied term proves that motion in one degree of freedom actively forces acceleration in the other. This algebraic relationship is validated by the nonlinear time-domain responses (Figs. 2 and 6), which visually capture this energy transfer: a perturbation entirely in the tilt angle actively accelerates the rotor as the system seeks to balance the internal gyroscopic torques.

When the system is linearized to evaluate stability near an operating point, these dynamic velocity cross-terms map directly into the damping matrix (C_0). As derived in Section VI-B4, the off-diagonal entries of the damping matrix contain terms that scale linearly with the rotor speed ($\pm J\Omega\sin(2\psi)$). Because these terms are skew-symmetric and proportional to Ω, they cause the natural frequencies of the coupled system to diverge as the turbine speeds up—a phenomenon classically known as frequency splitting or "whirling." The symbolic framework successfully isolates the mathematical root of this phenomenon: the rotating rotor acts as an active, speed-dependent gyroscopic spring that stiffens one mode branch (forward whirl) while softening the other (backward whirl).

C. Limitations of the Analytical Approach

While the symbolic Lagrangian methodology provides unparalleled transparency for identifying the root causes of aeroelastic phenomena, the reduced-order nature of these models inherently introduces specific mathematical and physical limitations.

The most prominent limitation encountered in this analysis is the boundary of the "frozen-azimuth" approximation. As demonstrated in the time-domain validation plots (Figs. 4 and 8), linearizing the EOMs requires freezing the rotor at a specific angular position ($\psi_0 = 0°$). Because the coupling terms in the generalized mass and stiffness matrices are highly sensitive to the trigonometric orientation of the blade, the constant-coefficient LTI system is only accurate for small perturbations near this equilibrium. As the true nonlinear rotor continues to spin, the local tangent approximation drifts, causing the linear simulation to rapidly diverge. In the 3-DOF model, this divergence occurs even faster due to the high-frequency sensitivity of the flexible blade mode. In practice, full-rotor aeroelastic analyses circumvent this limitation by applying the Multi-Blade Coordinate

(MBC) transformation (Coleman transformation), which converts the individual, periodically varying blade coordinates into constant, non-rotating reference frames.

Secondly, the analytical models isolated the structural dynamics from advanced aerodynamic loading and generator control systems. For instance, the purely real, positive eigenvalue ($\lambda = 1.0960$) extracted from both models indicates a static instability in the rotor azimuth mode. Physically, this is a mathematical artifact of modeling an un-driven, heavy rigid body as a free hinge; without aerodynamic thrust to drive it or generator torque to regulate it, gravity pulls the mass away from the vertical equilibrium.

Finally, the assumed modes method in this thesis utilized only a single flapwise shape function to discretize the blade. Modern multi-megawatt blades exhibit complex torsional and edgewise modes that couple tightly with flapwise bending, ultimately driving catastrophic instabilities such as classical flutter. While the symbolic models in this thesis serve as essential educational tools for proving the existence and mechanisms of multi-body energy transfer, capturing these high-order interactions across a fully spinning, fully aerodynamic rotor necessitates the use of high-fidelity, black-box numerical simulators like OpenFAST for comprehensive design load cases.

VIII. Conclusion

APPENDIX

A. Simple Pendulum

The inital step will be deriving the equiations of motion of a simple pendulum using Lagrangian mechanics. This will serve as a concept for the symbolic derivation process and allow for the validation of the method against known results. I will derive the equations for both a point mass suspended at the end of the pendulum and a rigid body pendulum.

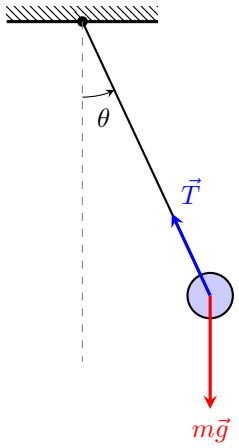


Fig. 9. Free Body Diagram of a simple pendulum with a point mass. The generalized coordinate θ is defined relative to the vertical, and the two primary forces are the tension $\vec{T}$ and gravity $m\vec{g}$, which contains a radial ($mg\cos\theta$) and tangential ($mg\sin\theta$) component.

Point Mass:

$$\ddot{\theta} + \frac{g}{l}\sin\theta = 0 \tag{29}$$

Rigid Body: For a uniform rod of mass m and length l, $I = \frac{1}{3}ml^2$. The resulting EOM is:

$$\ddot{\theta} + \frac{3g}{2l}\sin\theta = 0 \tag{30}$$

B. Double Pendulum

The same processes were applied to a double pendulum, two pendulums attached end to end. This systems is another well known system, making it another good proof of concept for the symbolic derivation process. The equations of motion for the double pendulum are more complex than the simple pendulum, as they involve the interaction between the two masses and their respective angles. The angles are θ_1 and θ_2 calculated relative to the previous angle.

Rigid Body EOM:

$$\begin{bmatrix} \frac{1}{3}l_1^2m_1 + l_1^2m_2 + l_1l_2m_2\cos\theta_2 + \frac{1}{3}l_2^2m_2 & l_2m_2\left(\frac{1}{2}l_1\cos\theta_2 + \frac{1}{3}l_2\right) \\ l_2m_2\left(\frac{1}{2}l_1\cos\theta_2 + \frac{1}{3}l_2\right) & \frac{1}{3}l_2^2m_2 \end{bmatrix} \ddot{q} = F \tag{31}$$

C. 2-DOF Rigid Nacelle-Rotor Model

A single blade model, working on increasing complexity, was derived using a 2-DOF "pendulum on a cart" analogy. This model consists of a translational degree of freedom x representing the tower's fore-aft motion, and a rotational degree of freedom ψ representing the azimuth of the blade. The mass is separated into a nacelle mass (m_n), and a rotor mass ($m_s + m_b$).

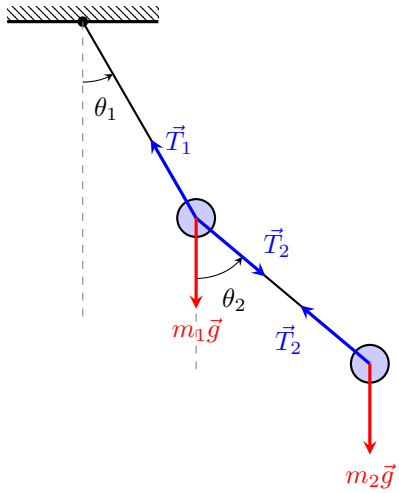


Fig. 10. Free Body Diagram of a double pendulum. The coordinates θ_1 and θ_2 are defined relative to the vertical. Mass m_1 is acted upon by gravity $m_1\vec{g}$ and the tensions from both rods, while m_2 is acted upon by $m_2\vec{g}$ and the tension $\vec{T}_2$.

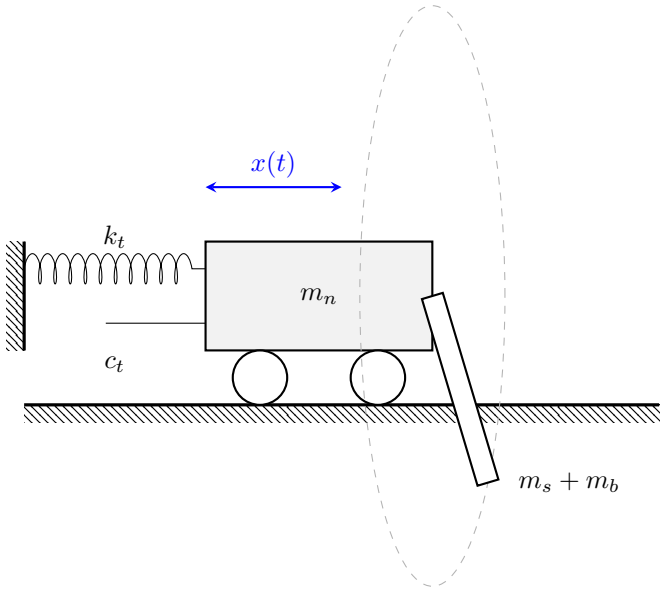


Fig. 11. Refined 2-DOF model with azimuthal rotation (ψ).

$$\begin{bmatrix} m_b + m_n + m_s & 0 \\ 0 & I_{rxx} + (Y_r^2 + Z_r^2)(m_b + m_s) \end{bmatrix} \ddot{q} = \begin{bmatrix} -c_t\dot{x} - k_t x \\ -g(m_b + m_s)(Y_r \cos\psi - Z_r \sin\psi) \end{bmatrix} \tag{32}$$

D. Fixed Nacelle, Flexible Blade

A flexible blade rotating about a fixed nacelle model was developed as a simplified model for flexible blade analysis. This model employs a flexing degree of freedom (q) to represent the deflection of the blade, and a rotational degree of freedom (ψ) representing the azimuth. The nacelle is assumed to be rigid and stationary for this derivation to isolate the rotation and blade bending.

The system's kinetic energy consists of rotational inertia for the hub and blade (J_{hub}, J_b) and flexural energy associated with the generalized modal mass (GM_b). Potential energy accounts for the modal elastic restoration of the blade (K_b) and

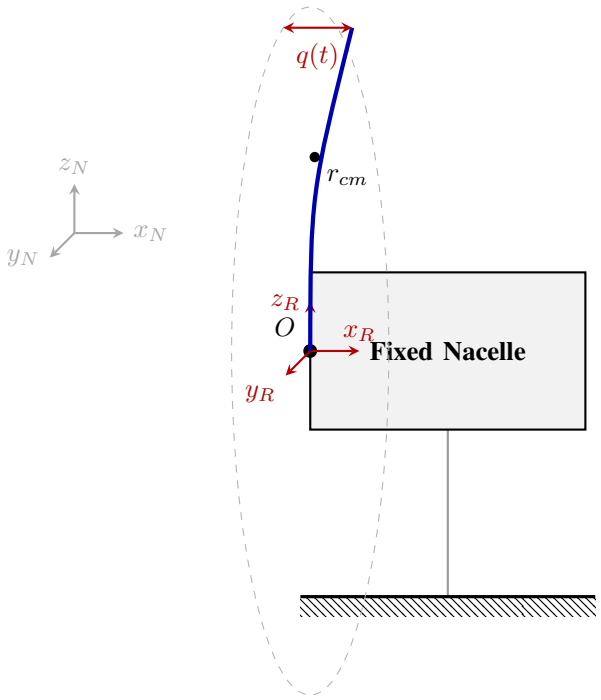


Fig. 12. Side-on perspective of the rotating flexible blade.

gravitational effects on the blade's center of mass (r_{cm}). The mass matrix M and forcing vector F cleanly map these dependencies:

$$M = \begin{bmatrix} GM_b & 0 \\ 0 & J_b + J_{hub} \end{bmatrix} \tag{33}$$

$$F = \begin{bmatrix} -K_b q(t) \\ g m_b r_{cm} \sin\psi(t) \end{bmatrix} \tag{34}$$

This model serves as the basis for calculating the blade's natural frequencies and analyzing how the gravitational load fluctuates as a function of the azimuth ψ.

E. Flexible Tower, Rigid Blade

To increase complexity the next model incorporates tower flexibility. The tower's bending is modeled as a single generalized coordinate (q_{T1}). This transition from a simple rigid tilt to a flexible representation involves the use of modal parameters, including the modal mass (GM_{T11}), modal stiffness (K_{eT11}), and modal damping (D_{eT11}).

To introduce structural coupling and enable the analysis of whirling phenomena, this model expands to include a flexible tower supporting a rotating blade. The tower's first fore-aft bending mode is modeled using the generalized coordinate q_{T1}. This model incorporates the tower's generalized modal mass (GM_{T11}), modal stiffness (k_t), and modal damping coefficient (d_t).The rotor remains a rigid body with an inertial frame R rotating at an azimuth ψ.

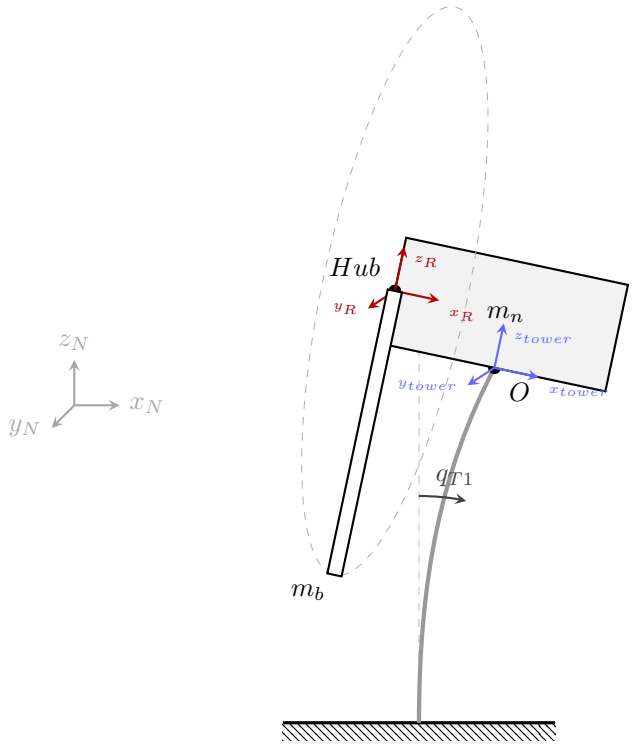


Fig. 13. Schematic of the Flexible Tower model with modal deflection q_{T1}.

The mass matrix M and forcing vector F accurately capture the trigonometric coupling terms (e.g., $X_h Z_b m_b \sin\psi$) generated by the non-linear interaction of the tower's lateral deflection and the rotor's dynamic azimuth position:

The equations of motion were derived by considering the kinetic energy of the flexible tower, the translating and rotating nacelle, and the rotating rotor. The mass matrix M and forcing vector F are expressed in their full symbolic form below.

$$M = \begin{bmatrix} GM_{T11} + J_{yyN} + J_{yyb}\cos^2\psi + J_{zzb}\sin^2\psi + m_b(X_h^2 + Z_b^2\cos^2\psi + 2Z_bZ_h\cos\psi + Z_h^2) + m_n(X_n^2 + Z_n^2) & X_hZ_bm_b\sin\psi \\ X_hZ_bm_b\sin\psi & J_{xxb} + Z_b^2m_b \end{bmatrix} \tag{35}$$

$$F = \begin{bmatrix} J_{yyb}\sin(2\psi)\dot{\psi}\dot{q}_{T1} - J_{zzb}\sin(2\psi)\dot{\psi}\dot{q}_{T1} + Z_bm_b(-X_h\cos\psi\dot{\psi} + Z_b\sin(2\psi)\dot{q}_{T1} + 2Z_h\sin\psi\dot{q}_{T1})\dot{\psi} - d_t\dot{q}_{T1} + gm_b(X_h\cos q_{T1} + Z_b\sin q_{T1}\cos\psi + Z_h\sin q_{T1}) + gm_n(X_n\cos q_{T1} + Z_n\sin q_{T1}) - k_tq_{T1} \\ \left(-J_{yyb}\cos\psi\dot{q}_{T1}^2 + J_{zzb}\cos\psi\dot{q}_{T1}^2 - Z_b^2m_b\cos\psi\dot{q}_{T1}^2 - Z_bZ_hm_b\dot{q}_{T1}^2 + Z_bgm_b\cos q_{T1}\right)\sin\psi \end{bmatrix} \tag{36}$$

onebladeTN

May 14, 2026

[1]:
```
import numpy as np                                      # numerical arrays and␣
↪operations
import sympy as sp                                      # symbolic math engine
from sympy.physics.mechanics import *                   # ReferenceFrame, Point,␣
↪Lagrangian, etc.
import scipy.linalg as la                               # eigenvalues, linear solvers
from scipy.integrate import solve_ivp                   # Runge-Kutta ODE integrator
import matplotlib.pyplot as plt                         # 2D plotting
init_vprinting()                                        # render SymPy output as␣
↪LaTeX in notebook
```

[2]:
```
theta, psi = dynamicsymbols('theta psi')                # DOFs: nacelle tilt theta␣
↪and rotor azimuth psi
t     = sp.symbols('t')                                 # independent time variable
Omega = sp.Symbol('Omega', real=True, positive=True) # symbolic steady-state␣
↪rotor speed

m_n, m_b         = sp.symbols('m_n m_b')                # nacelle and blade masses
k_t, c_t, g      = sp.symbols('k_t c_t g')              # tower stiffness, damping,␣
↪gravity
X_n, Z_n         = sp.symbols('X_n Z_n')                # nacelle COM position in␣
↪tower frame
X_h, Z_h         = sp.symbols('X_h Z_h')                # hub location in tower␣
↪frame
Z_b               = sp.symbols('Z_b')                    # blade COM radial offset␣
↪in rotor frame
J_xx_n, J_yy_n, J_zz_n = sp.symbols('J_xx_n J_yy_n J_zz_n')  # nacelle inertia␣
↪components
J_xx_b, J_yy_b, J_zz_b = sp.symbols('J_xx_b J_yy_b J_zz_b')  # blade inertia␣
↪components
```

[3]:
```
N = ReferenceFrame('N')                                 # inertial ground frame
T = N.orientnew('T', 'Axis', [theta, N.y])              # tower/nacelle frame:␣
↪tilts about N.y
R = T.orientnew('R', 'Axis', [psi,   T.x])              # rotor frame: spins about␣
↪T.x
```

```
O    = Point('O')                                          # fixed reference point at␣
 ↪tower top
NCOM = O.locatenew('NCOM', X_n*T.x + Z_n*T.z)          # nacelle COM position
Hub  = O.locatenew('Hub',  X_h*T.x + Z_h*T.z)          # hub center position
BCOM = Hub.locatenew('BCOM', Z_b*R.z)                  # blade COM position in␣
 ↪rotor frame

O.set_vel(N, 0)                                        # tower top is fixed in␣
 ↪inertial frame
NCOM.v2pt_theory(O,   N, T)                            # nacelle COM velocity via␣
 ↪two-point theorem
Hub.v2pt_theory(O,    N, T)                            # hub velocity via␣
 ↪two-point theorem
BCOM.v2pt_theory(Hub, N, R)                            # blade COM velocity via␣
 ↪two-point theorem
```

[3]: $Z_h\dot{\theta}\hat{\mathbf{t}}_\mathbf{x} - X_h\dot{\theta}\hat{\mathbf{t}}_\mathbf{z} + Z_b\cos(\psi)\dot{\theta}\hat{\mathbf{r}}_\mathbf{x} - Z_b\dot{\psi}\hat{\mathbf{r}}_\mathbf{y}$

[4]:
```
J_nac   = inertia(T, J_xx_n, J_yy_n, J_zz_n)         # nacelle inertia dyadic in␣
 ↪tower frame
J_blade = inertia(R, J_xx_b, J_yy_b, J_zz_b)          # blade inertia dyadic in␣
 ↪rotor frame

Nacelle = RigidBody('Nacelle', NCOM, T, m_n, (J_nac,   NCOM))  # nacelle rigid␣
 ↪body
Blade   = RigidBody('Blade',   BCOM, R, m_b, (J_blade, BCOM))  # blade rigid body
```

[5]:
```
Nacelle.potential_energy = (m_n*g*NCOM.pos_from(O).dot(N.z)     # nacelle␣
 ↪gravitational PE
                            + sp.Rational(1,2)*k_t*theta**2)    # tower␣
 ↪torsional spring PE
Blade.potential_energy   = m_b*g*BCOM.pos_from(O).dot(N.z)      # blade␣
 ↪gravitational PE

damping = [(T, -c_t*theta.diff(t)*N.y)]                         # viscous tower␣
 ↪damping torque
```

[17]:
```
L   = Lagrangian(N, Nacelle, Blade)
print("Lagrangian")              # T - V system Lagrangian
display(L)                                             # show symbolic Lagrangian
LM  = LagrangesMethod(L, [theta, psi], frame=N, forcelist=damping)  # Lagrange␣
 ↪method with damping
eom = LM.form_lagranges_equations()
print("Equations of Motion")             # derive 2-DOF equations of motion
display(eom)                                           # show symbolic␣
 ↪equations of motion
```

Lagrangian

$$\frac{J_{xxb}\dot{\psi}^2}{2} + \frac{J_{yyn}\dot{\theta}^2}{2} - gm_b\left(-X_h\sin(\theta) + Z_b\cos(\psi)\cos(\theta) + Z_h\cos(\theta)\right) -$$
$$gm_n\left(-X_n\sin(\theta) + Z_n\cos(\theta)\right) - \frac{k_t\theta^2}{2} + \frac{m_b\left(X_h^2\dot{\theta}^2 + 2X_hZ_b\sin(\psi)\dot{\psi}\dot{\theta} + Z_b^2\cos^2(\psi)\dot{\theta}^2 + Z_b^2\dot{\psi}^2 + 2Z_bZ_h\cos(\psi)\dot{\theta}^2 + Z\right.}{2}$$
$$\frac{m_n\left(X_n^2\dot{\theta}^2 + Z_n^2\dot{\theta}^2\right)}{2} + \frac{\left(J_{yyb}\cos^2(\psi)\dot{\theta} + J_{zzb}\sin^2(\psi)\dot{\theta}\right)\dot{\theta}}{2}$$

Equations of Motion

$$\left[-J_{yyb}\sin(\psi)\cos(\psi)\dot{\psi}\dot{\theta} + \frac{J_{yyb}\cos^2(\psi)\ddot{\theta}}{2} + J_{yyn}\ddot{\theta} + \frac{J_{zzb}\sin^2(\psi)\ddot{\theta}}{2} + J_{zzb}\sin(\psi)\cos(\psi)\dot{\psi}\dot{\theta} + c_t\dot{\theta} + gm_b\left(-X_h\cos(\theta) - Z\right.\right.$$
$$\left.\left.J_{xxb}\ddot{\psi} - Z\right.\right.$$

[7]:
```
M = sp.simplify(sp.trigsimp(LM.mass_matrix))          # symbolic mass matrix M(q)
F = sp.simplify(sp.trigsimp(LM.forcing))               # symbolic forcing vector␣
↪F(q, qdot)
print("Mass Matrix M:")
display(M)
print("Forcing Vector F:")
display(F)
```

Mass Matrix M:

$$\begin{bmatrix} J_{yyb}\cos^2(\psi) + J_{yyn} + J_{zzb}\sin^2(\psi) + m_b\left(X_h^2 + Z_b^2\cos^2(\psi) + 2Z_bZ_h\cos(\psi) + Z_h^2\right) + m_n\left(X_n^2 + Z_n^2\right) & X_hZ_bm_b\sin(\\ X_hZ_bm_b\sin(\psi) & J_{xxb} + Z_b^2m \end{bmatrix}$$

Forcing Vector F:

$$\begin{bmatrix} J_{yyb}\sin(2\psi)\dot{\psi}\dot{\theta} - J_{zzb}\sin(2\psi)\dot{\psi}\dot{\theta} - X_hZ_bm_b\cos(\psi)\dot{\psi}^2 + X_hgm_b\cos(\theta) + X_ngm_n\cos(\theta) + Z_b^2m_b\sin(2\psi)\dot{\psi}\dot{\theta} + 2Z_t \\ \left(-J_{yyb}\cos(\psi)\dot{\theta}^2 + J_{zzb}\cos(\psi)\dot{\theta}^2 - Z_b^2m_b\cos(\psi)\dot{\theta}^2 - Z\right. \end{bmatrix}$$

0.1 Nonlinear Simulation

[8]:
```
params = {
    m_n: 50000,                                        # nacelle mass [kg]
    m_b: 10000,                                        # blade mass [kg]
    X_n: -1.5,                                         # nacelle COM x-offset [m]
    Z_n:  2.0,                                         # nacelle COM z-offset [m]
    X_h: -2.0,                                         # hub x-offset [m]
    Z_h:  3.0,                                         # hub z-offset [m]
    Z_b:  1.5,                                         # blade COM radial offset␣
↪[m]
    k_t:  1e7,                                         # tower torsional␣
↪stiffness [N.m/rad]
    c_t:  5e5,                                         # tower torsional damping␣
↪[N.m.s/rad]
```

```
    g:    9.81,                                          # gravitational␣
↪acceleration [m/s^2]
    J_xx_n: 1e6, J_yy_n: 5e6, J_zz_n: 1e6,          # nacelle inertia components␣
↪[kg.m^2]
    J_xx_b: 1e5, J_yy_b: 1e6, J_zz_b: 1e6,          # blade inertia components␣
↪[kg.m^2]
}
OMEGA_VAL = 1.5                                       # operating rotor speed␣
↪[rad/s]
```

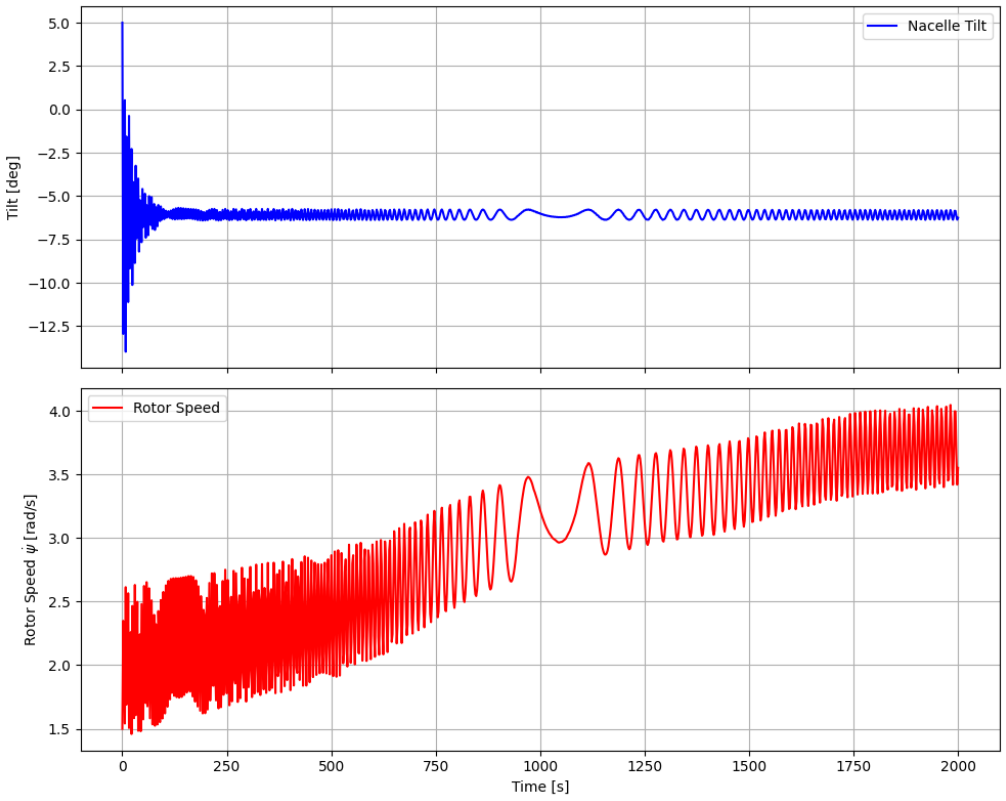
[9]:
```
q_sym     = [theta, psi]                              # generalized coordinates
dq_sym    = [theta.diff(t), psi.diff(t)]              # generalized velocities
state_vec = q_sym + dq_sym                            # full state: [theta, psi,␣
↪theta_dot, psi_dot]

M_func = sp.lambdify((state_vec,), M.subs(params), 'numpy')  # numerical mass␣
↪matrix callable
F_func = sp.lambdify((state_vec,), F.subs(params), 'numpy')  # numerical forcing␣
↪vector callable

def system_dynamics(t, y):
    M_num  = M_func(y)                                # mass matrix at current␣
↪state
    F_num  = F_func(y).flatten()                      # forcing vector at current␣
↪state
    accels = la.solve(M_num, F_num)                   # solve M*q_ddot = F for␣
↪accelerations
    return [y[2], y[3], accels[0], accels[1]]         # return [theta_dot,␣
↪psi_dot, theta_ddot, psi_ddot]
```

[10]:
```
t_span = (0, 2000)                                    # simulation window [s]
t_eval = np.linspace(*t_span, 1000)                   # output time grid
y0     = [np.deg2rad(5), 0, 0, OMEGA_VAL]             # IC: [5 deg tilt, 0␣
↪azimuth, 0 tilt rate, Omega]

sol = solve_ivp(system_dynamics, t_span, y0, t_eval=t_eval, method='RK45')  #␣
↪RK45 integration
print('Simulation complete.')
```

```
Simulation complete.
```

[11]:
```
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 8), sharex=True)  # two-panel␣
↪figure

ax1.plot(sol.t, np.rad2deg(sol.y[0]), 'b', label='Nacelle Tilt')     # tilt in␣
↪degrees
ax1.set_ylabel('Tilt [deg]')
```

```
ax1.legend(); ax1.grid(True)

ax2.plot(sol.t, sol.y[3], 'r', label='Rotor Speed')                    # speed in␣
↪rad/s
ax2.set_ylabel('Rotor Speed $\\dot{\\psi}$ [rad/s]')
ax2.set_xlabel('Time [s]')
ax2.legend(); ax2.grid(True)

plt.tight_layout(); plt.show()
```



0.2 Linearization

[12]:
```
op_point = {
    theta:              0,       # linearize at zero tilt
    theta.diff(t):      0,       # zero tilt rate at equilibrium
    theta.diff(t, 2): 0,         # zero tilt acceleration at equilibrium
    psi.diff(t):        Omega,   # steady-state rotor speed
    psi.diff(t, 2):     0,       # zero rotor acceleration at equilibrium
}
```

```
q_vec  = sp.Matrix([theta, psi])                        # coordinate column vector
dq_vec = q_vec.diff(t)                                  # velocity column vector

M_lin = sp.simplify(LM.mass_matrix.subs(op_point))    # linearized mass matrix
f_vec = LM.forcing                                      # symbolic forcing vector
C_lin = sp.simplify(-f_vec.jacobian(dq_vec).subs(op_point))  # damping /␣
↪gyroscopic matrix
K_lin = sp.simplify(-f_vec.jacobian(q_vec).subs(op_point))   # stiffness /␣
↪centrifugal matrix

display(M_lin); display(C_lin); display(K_lin)
```

$$\begin{bmatrix} J_{yyb}\cos^2(\psi) + J_{yyn} + J_{zzb}\sin^2(\psi) + m_b\left(X_h^2 + Z_b^2\cos^2(\psi) + 2Z_bZ_h\cos(\psi) + Z_h^2\right) + m_n\left(X_n^2 + Z_n^2\right) & X_hZ_bm_b\sin(\\ X_hZ_bm_b\sin(\psi) & J_{xxb} + Z_b^2m \end{bmatrix}$$

$$\begin{bmatrix} -J_{yyb}\Omega\sin(2\psi) + J_{zzb}\Omega\sin(2\psi) - 2\Omega Z_bm_b\left(Z_b\cos(\psi) + Z_h\right)\sin(\psi) + c_t & 2\Omega X_hZ_bm_b\cos(\psi) \\ 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} -Z_ngm_n - gm_b\left(Z_b\cos(\psi) + Z_h\right) + k_t & -\Omega^2X_hZ_bm_b\sin(\psi) \\ 0 & -Z_bgm_b\cos(\psi) \end{bmatrix}$$

0.3 Eigenvalue Analysis

[13]:
```
M_sym, A_sym, _, _ = LM.linearize(                  # extract un-inverted␣
↪linearized matrices
    q_ind=[theta, psi],
    qd_ind=[theta.diff(t), psi.diff(t)],
    op_point=op_point
)

subs_dict = {**params, psi: 0, Omega: OMEGA_VAL}      # physical params +␣
↪operating point values
M_sub = M_sym.subs(subs_dict)                         # substitute into mass␣
↪matrix
A_sub = A_sym.subs(subs_dict)                         # substitute into␣
↪uninverted A matrix

A_ss  = sp.expand(M_sub.inv() * A_sub).evalf()        # state-space A = M^-1 *␣
↪A_uninv
display(A_ss)

A_num = np.array(A_ss.tolist(), dtype=np.float64)     # convert to NumPy for␣
↪eigenanalysis
eigenvalues, _ = la.eig(A_num)                        # compute eigenvalues

print(f'Operating Speed: Omega = {OMEGA_VAL} rad/s\n')
for i, lam in enumerate(eigenvalues):
```

```
    if np.abs(lam) > 1e-4 and np.imag(lam) >= 0:      # skip near-zero and␣
↪conjugate duplicates
        wn   = np.abs(lam)                            # natural frequency [rad/s]
        zeta = -np.real(lam) / wn                     # damping ratio
        print(f'Mode {i//2+1}: lambda={lam:.4f}  wn={wn/(2*np.pi):.4f} Hz ␣
↪zeta={zeta:.4f}')
```

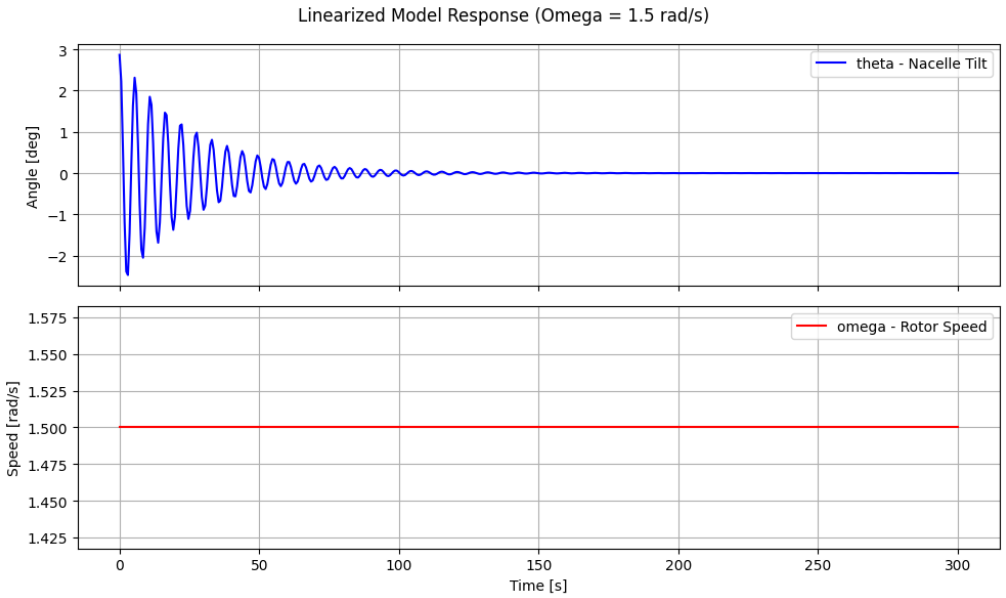
$$\begin{bmatrix} 0 & 0 & 1.0 & 0 \\ 0 & 0 & 0 & 1.0 \\ -1.30855072463768 & 0 & -0.07627765064836 & 0.0137299771167048 \\ 0 & 1.20122448979592 & 0 & 0 \end{bmatrix}$$

```
Operating Speed: Omega = 1.5 rad/s

Mode 1: lambda=-0.0381+1.1433j  wn=0.1821 Hz  zeta=0.0333
Mode 2: lambda=-1.0960+0.0000j  wn=0.1744 Hz  zeta=1.0000
Mode 2: lambda=1.0960+0.0000j  wn=0.1744 Hz  zeta=-1.0000
```

0.4 Linearized Simulation

[14]:
```
def lin_dynamics(t, x):
    return A_num @ x                                  # linear state-space:␣
↪x_dot = A*x

x0     = np.array([0.05, 0.0, 0.0, 0.0])              # 0.05 rad initial tilt␣
↪perturbation
t_span = (0, 300)
t_eval = np.linspace(*t_span, 500)

sol_lin = solve_ivp(lin_dynamics, t_span, x0, t_eval=t_eval, method='RK45')  #␣
↪integrate linear model

theta_abs   = sol_lin.y[0]                            # tilt perturbation␣
↪(op-point = 0, so delta_theta = theta)
psi_dot_abs = OMEGA_VAL + sol_lin.y[3]                # absolute rotor speed:␣
↪Omega + delta_psi_dot

fig, axs = plt.subplots(2, 1, figsize=(10, 6), sharex=True)

axs[0].plot(sol_lin.t, np.rad2deg(theta_abs), 'b', label='theta - Nacelle Tilt')
axs[0].set_ylabel('Angle [deg]'); axs[0].legend(); axs[0].grid(True)

axs[1].plot(sol_lin.t, psi_dot_abs, 'r', label='omega - Rotor Speed')
axs[1].set_ylabel('Speed [rad/s]'); axs[1].set_xlabel('Time [s]')
axs[1].legend(); axs[1].grid(True)

plt.suptitle(f'Linearized Model Response (Omega = {OMEGA_VAL} rad/s)')
```

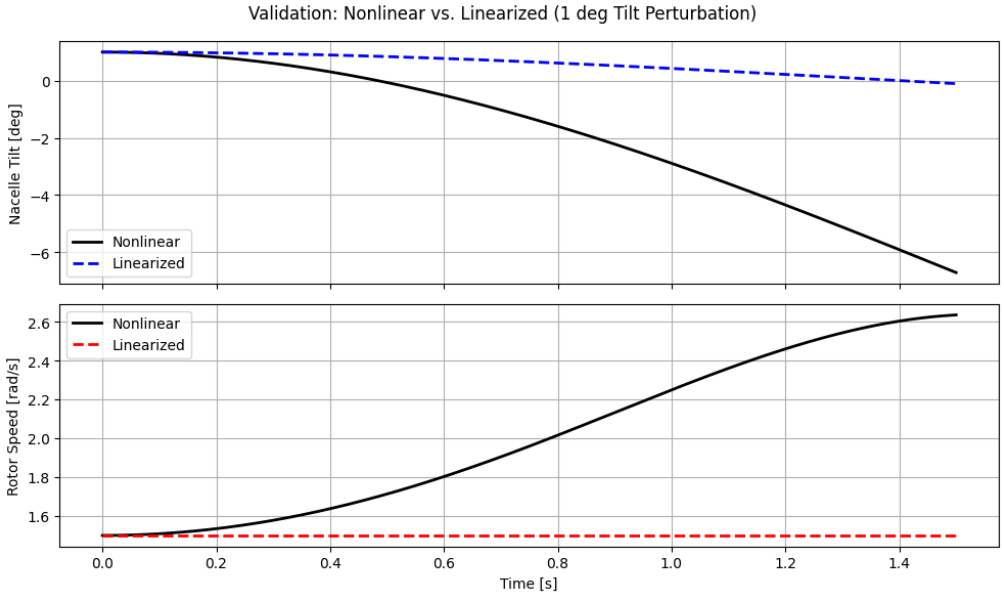
```
plt.tight_layout(); plt.show()
```



0.5 Validate Linearization against Nonlinear Model

[15]:
```
theta_pert = np.deg2rad(1.0)                          # 1 degree tilt perturbation
init_nl    = [theta_pert, 0.0, 0.0, OMEGA_VAL]        # nonlinear ICs: absolute␣
↪states
init_lin   = [theta_pert, 0.0, 0.0, 0.0]              # linear ICs: perturbations␣
↪from equilibrium

t_span = (0, 1.5)                                      # short window to isolate␣
↪transient
t_eval = np.linspace(*t_span, 500)

sol_nl  = solve_ivp(system_dynamics, t_span, init_nl,  t_eval=t_eval,␣
↪method='RK45')  # nonlinear
sol_lin = solve_ivp(lin_dynamics,    t_span, init_lin, t_eval=t_eval,␣
↪method='RK45')  # linearized

theta_nl_deg  = np.rad2deg(sol_nl.y[0])               # nonlinear tilt [deg]
theta_lin_deg = np.rad2deg(sol_lin.y[0])              # linear tilt perturbation␣
↪[deg]
psi_d_nl      = sol_nl.y[3]                           # nonlinear rotor speed␣
↪[rad/s]
```

```
psi_d_lin     = OMEGA_VAL + sol_lin.y[3]             # linear absolute rotor␣
↪speed [rad/s]

fig, axs = plt.subplots(2, 1, figsize=(10, 6), sharex=True)

axs[0].plot(sol_nl.t,  theta_nl_deg,  'k-',  lw=2, label='Nonlinear')
axs[0].plot(sol_lin.t, theta_lin_deg, 'b--', lw=2, label='Linearized')
axs[0].set_ylabel('Nacelle Tilt [deg]'); axs[0].legend(); axs[0].grid(True)

axs[1].plot(sol_nl.t,  psi_d_nl,  'k-',  lw=2, label='Nonlinear')
axs[1].plot(sol_lin.t, psi_d_lin, 'r--', lw=2, label='Linearized')
axs[1].set_ylabel('Rotor Speed [rad/s]'); axs[1].set_xlabel('Time [s]')
axs[1].legend(); axs[1].grid(True)

plt.suptitle('Validation: Nonlinear vs. Linearized (1 deg Tilt Perturbation)')
plt.tight_layout(); plt.show()
```



onebladeTNflex

May 14, 2026

[1]:
```
import numpy as np                                    # numerical arrays and␣
↪operations
import sympy as sp                                    # symbolic math engine
from sympy.physics.mechanics import * # ReferenceFrame, Point, Lagrangian, etc.
import scipy.linalg as la                             # eigenvalues, linear solvers
from scipy.integrate import solve_ivp                 # Runge-Kutta ODE integrator
import matplotlib.pyplot as plt                       # 2D plotting
init_vprinting()                                      # render SymPy output as␣
↪LaTeX in notebook
```

1 3 DOF

1.1 Tilting Nacelle, Single Flexible Blade

[2]:
```
# Symbolic variables
theta, psi, q    = dynamicsymbols('theta psi q')      # DOFs: nacelle tilt, rotor␣
↪azimuth, blade flap
t                = sp.symbols('t')                    # independent time variable
Omega            = sp.Symbol('Omega', real=True)      # symbolic steady-state␣
↪rotor speed

m_n, m_b         = sp.symbols('m_n m_b')              # nacelle and blade masses
k_t, c_t, g      = sp.symbols('k_t c_t g')            # tower stiffness, damping,␣
↪gravity
X_n, Z_n         = sp.symbols('X_n Z_n')              # nacelle COM position in␣
↪tower frame
X_h, Z_h         = sp.symbols('X_h Z_h')              # hub location in tower␣
↪frame
Z_b              = sp.symbols('Z_b')                  # blade COM radial offset␣
↪in rotor frame
J_xx_n, J_yy_n, J_zz_n = sp.symbols('J_xx_n J_yy_n J_zz_n')  # nacelle inertia␣
↪components
J_xx_b, J_yy_b, J_zz_b = sp.symbols('J_xx_b J_yy_b J_zz_b')  # blade inertia␣
↪components

GM_b, K_b        = sp.symbols('GM_b K_b')             # blade modal mass and␣
↪modal stiffness
```

```
M_phi, M_rphi    = sp.symbols('M_phi M_rphi')        # coupling integrals:␣
↪φ(r)dm and rφ(r)dm
```

[3]:
```
# Reference frames and kinematics
N = ReferenceFrame('N')                              # inertial ground frame
T = N.orientnew('T', 'Axis', [theta, N.y])           # tower/nacelle frame:␣
↪tilts about N.y
R = T.orientnew('R', 'Axis', [psi,   T.x])           # rotor frame: spins about␣
↪T.x

O    = Point('O')                                    # fixed reference point at␣
↪tower top
NCOM = O.locatenew('NCOM', X_n*T.x + Z_n*T.z)        # nacelle COM position
Hub  = O.locatenew('Hub',  X_h*T.x + Z_h*T.z)        # hub center position
BCOM_rigid = Hub.locatenew('BCOM_rigid', Z_b*R.z)    # undeformed blade COM␣
↪position

O.set_vel(N, 0)                                      # tower top is fixed in␣
↪inertial frame
NCOM.v2pt_theory(O,   N, T)                          # nacelle COM velocity via␣
↪two-point theorem
Hub.v2pt_theory(O,    N, T)                          # hub velocity via␣
↪two-point theorem
BCOM_rigid.v2pt_theory(Hub, N, R)                    # undeformed blade COM␣
↪velocity
```

[3]:
$$Z_h\dot{\theta}\hat{\mathbf{t}}_\mathbf{x} - X_h\dot{\theta}\hat{\mathbf{t}}_\mathbf{z} + Z_b\cos(\psi)\dot{\theta}\hat{\mathbf{r}}_\mathbf{x} - Z_b\dot{\psi}\hat{\mathbf{r}}_\mathbf{y}$$

[4]:
```
# Inertias and Rigid Bodies
J_nac   = inertia(T, J_xx_n, J_yy_n, J_zz_n)         # nacelle inertia dyadic in␣
↪tower frame
J_blade = inertia(R, J_xx_b, J_yy_b, J_zz_b)         # blade inertia dyadic in␣
↪rotor frame

Nacelle = RigidBody('Nacelle', NCOM, T, m_n, (J_nac, NCOM))                  #␣
↪nacelle rigid body
Blade   = RigidBody('Blade', BCOM_rigid, R, m_b, (J_blade, BCOM_rigid))      #␣
↪blade rigid body
```

[5]:
```
# Energies, Forces
# Potential Energies (Gravity + Strain)
Nacelle.potential_energy = (m_n*g*NCOM.pos_from(O).dot(N.z)     # nacelle␣
↪gravitational PE
                            + sp.Rational(1,2)*k_t*theta**2)    # tower torsional␣
↪spring PE
Blade.potential_energy   = m_b*g*BCOM_rigid.pos_from(O).dot(N.z) # blade␣
↪gravitational PE
```

```
V_flex = sp.Rational(1, 2) * K_b * q**2                          # blade flapwise␣
 ↪strain PE

# Flexible Kinetic Energy and FFRF Coupling
T_flex = sp.Rational(1, 2) * GM_b * q.diff(t)**2                 # blade flapwise␣
 ↪kinetic energy
v_hub = Hub.vel(N)
omega_b = R.ang_vel_in(N)
T_coupling = (q.diff(t) * M_phi * v_hub.dot(R.x)                 # flex␣
 ↪translation coupling
              + q.diff(t) * M_rphi * omega_b.dot(R.y))           # flex rotation␣
 ↪coupling
damping = [(T, -c_t*theta.diff(t)*N.y)]                          # viscous tower␣
 ↪damping torque

# display(Lagrangian(N, Nacelle))
# display(Lagrangian(N, Blade))
```

[6]:
```
# Equations of Motion via Lagrange's Method
L_rigid = Lagrangian(N, Nacelle, Blade)                          # rigid body␣
 ↪lagrangian
L = L_rigid + T_flex + T_coupling - V_flex                       # total system␣
 ↪lagrangian
print("Lagrangian:")
display(L) # Display the total Lagrangian
LM = LagrangesMethod(L, [theta, psi, q], forcelist=damping, frame=N)
EOM = LM.form_lagranges_equations()       # generate the symbolic EOMs
print("Equations of Motion:")
display(EOM)
```

Lagrangian:

$$\frac{GM_b\dot{q}^2}{2} + \frac{J_{xxb}\dot{\psi}^2}{2} + \frac{J_{yyn}\dot{\theta}^2}{2} - \frac{K_bq^2}{2} + M_\phi Z_h\dot{q}\dot{\theta} + M_{rphi}\cos(\psi)\dot{q}\dot{\theta} - gm_b\left(-X_h\sin(\theta) + Z_b\cos(\psi)\cos(\theta) + Z_h\cos(\theta)\right) - gm_n\left(-X_n\sin(\theta) + Z_n\cos(\theta)\right) - \frac{k_t\theta^2}{2} + \frac{m_b\left(X_h^2\dot{\theta}^2 + 2X_hZ_b\sin(\psi)\dot{\psi}\dot{\theta} + Z_b^2\cos^2(\psi)\dot{\theta}^2 + Z_b^2\dot{\psi}^2 + 2Z_bZ_h\cos(\psi)\dot{\theta}^2 + Z_h^2\dot{\theta}^2\right)}{2} + \frac{m_n\left(X_n^2\dot{\theta}^2 + Z_n^2\dot{\theta}^2\right)}{2} + \frac{\left(J_{yyb}\cos^2(\psi)\dot{\theta} + J_{zzb}\sin^2(\psi)\dot{\theta}\right)\dot{\theta}}{2}$$

Equations of Motion:

$$\begin{bmatrix} -J_{yyb}\sin(\psi)\cos(\psi)\dot{\psi}\dot{\theta} + \frac{J_{yyb}\cos^2(\psi)\ddot{\theta}}{2} + J_{yyn}\ddot{\theta} + \frac{J_{zzb}\sin^2(\psi)\ddot{\theta}}{2} + J_{zzb}\sin(\psi)\cos(\psi)\dot{\psi}\dot{\theta} + M_\phi Z_h\ddot{q} - M_{rphi}\sin(\psi)\dot{\psi}\dot{q} + \end{bmatrix}$$

[7]:
```
# Output
M = sp.simplify(sp.trigsimp(LM.mass_matrix))          # symbolic mass matrix M(q)
F = sp.simplify(sp.trigsimp(LM.forcing))               # symbolic forcing vector␣
 ↪F(q, qdot)
E = sp.simplify(sp.trigsimp(LM.eom))                   # symbolic equations of␣
 ↪motion E(q, qdot, qddot) = 0
print("Mass Matrix M:")
display(M)
print("Forcing Vector F:")
display(F)
print("Equations of Motion:")
display(E)
```

Mass Matrix M:

$$\begin{bmatrix} J_{yyb}\cos^2(\psi) + J_{yyn} + J_{zzb}\sin^2(\psi) + m_b\left(X_h^2 + Z_b^2\cos^2(\psi) + 2Z_bZ_h\cos(\psi) + Z_h^2\right) + m_n\left(X_n^2 + Z_n^2\right) & X_hZ_bm_b\sin \\ X_hZ_bm_b\sin(\psi) & J_{xxb} + Z_b^2n \\ M_\phi Z_h + M_{rphi}\cos(\psi) & 0 \end{bmatrix}$$

Forcing Vector F:

$$\begin{bmatrix} J_{yyb}\sin(2\psi)\dot{\psi}\dot{\theta} - J_{zzb}\sin(2\psi)\dot{\psi}\dot{\theta} + M_{rphi}\sin(\psi)\dot{\psi}\dot{q} - X_hZ_bm_b\cos(\psi)\dot{\psi}^2 + X_hgm_b\cos(\theta) + X_ngm_n\cos(\theta) + Z_b^2n \\ \left(-J_{yyb}\cos(\psi)\dot{\theta}^2 + J_{zzb}\cos(\psi)\dot{\theta}^2 - M_{rphi}\dot{q}\dot{\theta} - Z_b^2n \\ -K_bq + M_{rp} \end{bmatrix}$$

Equations of Motion:

$$\begin{bmatrix} -J_{yyb}\sin(2\psi)\dot{\psi}\dot{\theta} + \frac{J_{yyb}\cos(2\psi)\ddot{\theta}}{2} + \frac{J_{yyb}\ddot{\theta}}{2} + J_{yyn}\ddot{\theta} + J_{zzb}\sin(2\psi)\dot{\psi}\dot{\theta} - \frac{J_{zzb}\cos(2\psi)\ddot{\theta}}{2} + \frac{J_{zzb}\ddot{\theta}}{2} + M_\phi Z_h\ddot{q} - M_{rphi}\sin(\psi)\dot{\psi} \\ \end{bmatrix}$$

[8]:
```
params = {
    g: 9.81,
    m_n: 50000,
    m_b: 10000,

    J_xx_n: 1e6,
    J_yy_n: 5e6,
    J_zz_n: 1e6,

    J_xx_b: 1e5,
    J_yy_b: 1e6,
    J_zz_b: 1e6,

    X_n: -1.5,
    Z_n: 2.0,
    X_h: -2.0,
    Z_h: 3.0,
```

```
    Z_b: 1.5,

    k_t: 1e7,
    c_t: 5e5,

    GM_b: 8000,
    K_b: 1e7,
    M_phi: 5000,
    M_rphi: 60000
}
```

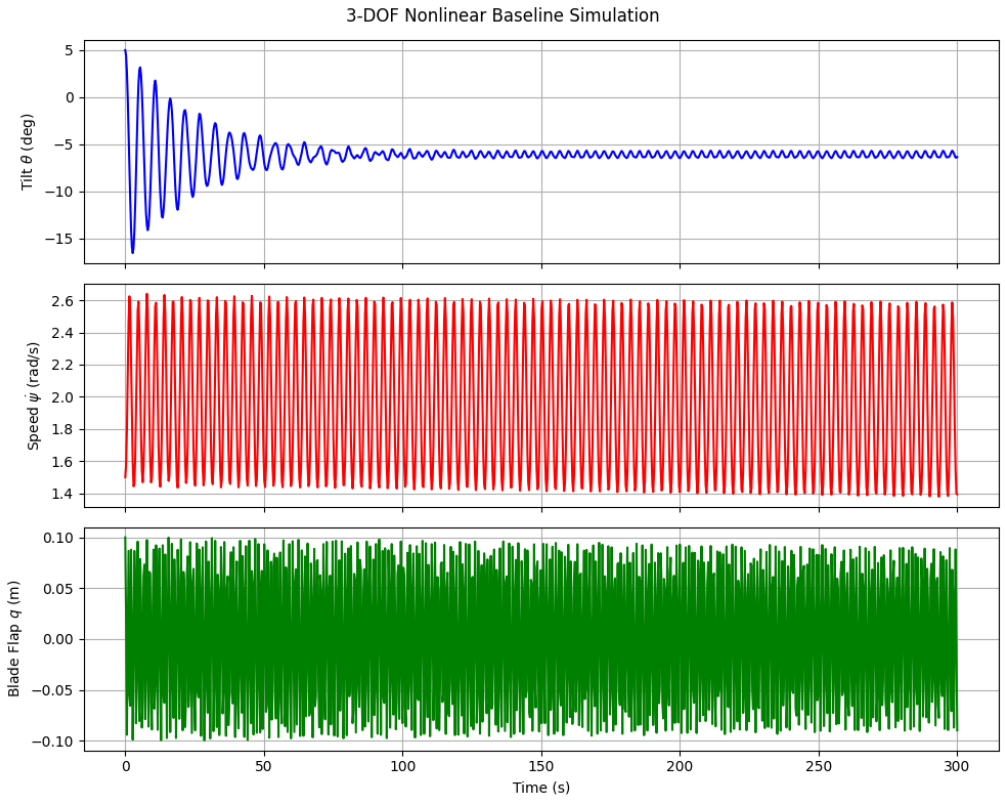
[9]:
```
# ========================================
# Numerical Mass and Forcing Matrices
# ========================================
M_sub = LM.mass_matrix.subs(params)
F_sub = LM.forcing.subs(params)

# State vector
states = [theta, psi, q, theta.diff(), psi.diff(), q.diff()]

# Lambdified functions
M_func = sp.lambdify((states,), M_sub, modules='numpy')
F_func = sp.lambdify((states,), F_sub, modules='numpy')

# ========================================
# 3-DOF Nonlinear ODE
# ========================================
def ode_3dof(t, y):

    # Evaluate matrices
    M_num = M_func(y)
    F_num = F_func(y)

    # Solve for accelerations
    accels = np.linalg.solve(M_num, F_num).flatten()

    return [
        y[3],          # theta_dot
        y[4],          # psi_dot
        y[5],          # q_dot
        accels[0],     # theta_ddot
        accels[1],     # psi_ddot
        accels[2]      # q_ddot
    ]
```

[10]:
```
# ========================================
# Initial Conditions
```

```
# ========================================
# [theta, psi, q, theta_dot, psi_dot, q_dot]
init_3dof = [
    np.deg2rad(5),    # 5 deg tilt
    0.0,              # azimuth
    0.1,              # flap displacement
    0.0,              # tilt rate
    1.5,              # rotor speed
    0.0               # flap velocity
]

# ========================================
# Run Simulation
# ========================================
t_span = (0, 300)
t_eval = np.linspace(*t_span, 1000)

sol = solve_ivp(ode_3dof,t_span,init_3dof,t_eval=t_eval,method='RK45')

# ========================================
# Print Simulation Information
# ========================================
print("Simulation Complete")
print(f"Number of Time Steps: {len(sol.t)}")
print(f"Final Tilt Angle: {np.rad2deg(sol.y[0,-1]):.3f} deg")
print(f"Final Rotor Speed: {sol.y[4,-1]:.3f} rad/s")
print(f"Final Blade Flap: {sol.y[2,-1]:.3f} m")

# ========================================
# Plot Results
# ========================================
fig, (ax1, ax2, ax3) = plt.subplots(3,1,figsize=(10, 8),sharex=True)

# Tilt response
ax1.plot(sol.t, np.rad2deg(sol.y[0]), 'b')
ax1.set_ylabel(r'Tilt $\theta$ (deg)')
ax1.grid(True)

# Rotor speed
ax2.plot(sol.t, sol.y[4], 'r')
ax2.set_ylabel(r'Speed $\dot{\psi}$ (rad/s)')
ax2.grid(True)

# Blade flap response
ax3.plot(sol.t, sol.y[2], 'g')
ax3.set_ylabel(r'Blade Flap $q$ (m)')
ax3.set_xlabel('Time (s)')
```

```
ax3.grid(True)

plt.suptitle('3-DOF Nonlinear Baseline Simulation')

plt.tight_layout()
plt.show()
```

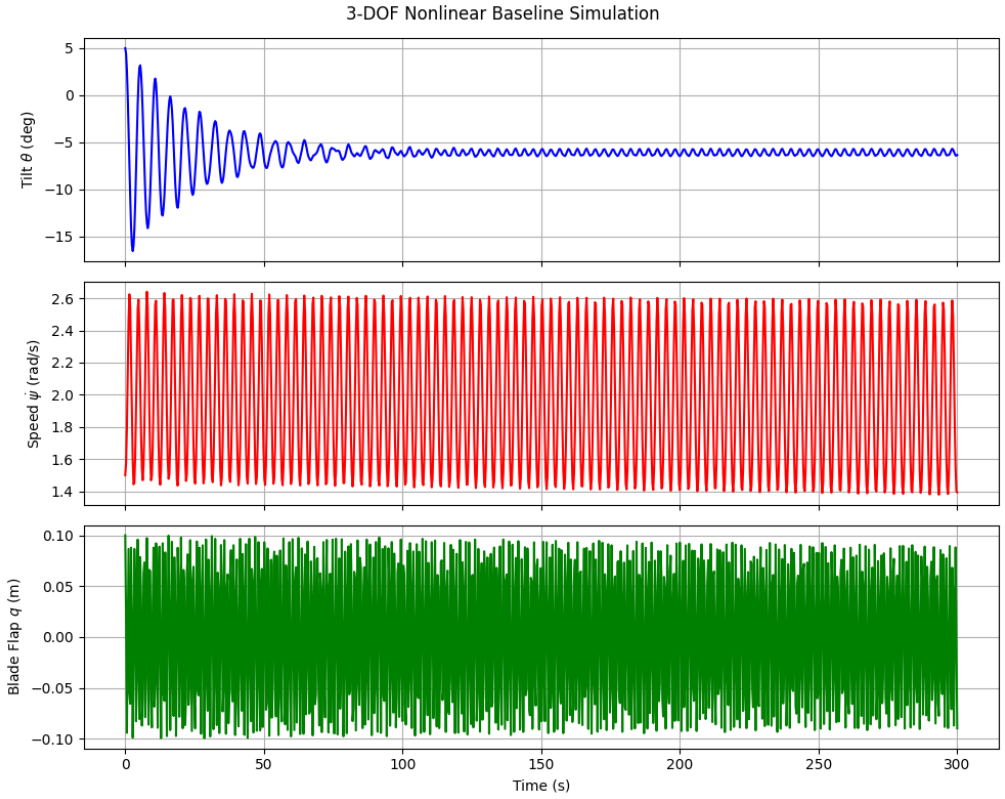
```
Simulation Complete
Number of Time Steps: 1000
Final Tilt Angle: -6.343 deg
Final Rotor Speed: 1.394 rad/s
Final Blade Flap: -0.090 m
```



[11]:
```
# ========================================
# Parameters for 3-DOF
# ========================================

# 2. Lambdify the 3rd DOF
M_sub = LM.mass_matrix.subs(params)
```

```
F_sub = LM.forcing.subs(params)

# Define the 6-element state vector: [pos1, pos2, pos3, vel1, vel2, vel3]
states = [theta, psi, q, theta.diff(), psi.diff(), q.diff()]

M_func = sp.lambdify((states,), M_sub, modules='numpy')
F_func = sp.lambdify((states,), F_sub, modules='numpy')

# ==========================================
# 3. 3-DOF ODE Function
# ==========================================
def ode_3dof(t, y):
    # y = [theta, psi, q, theta_dot, psi_dot, q_dot]

    # Solve M(q) * q_ddot = F(q, q_dot)
    M_num = M_func(y)
    F_num = F_func(y)

    # Extract accelerations [theta_ddot, psi_ddot, q_ddot]
    accels = np.linalg.solve(M_num, F_num).flatten()

    # Return [vel1, vel2, vel3, acc1, acc2, acc3]
    return [y[3], y[4], y[5], accels[0], accels[1], accels[2]]

# ==========================================
# 4. Run Simulation
# ==========================================
# Init: 5 deg tilt, 0 azimuth, 0.1m deflection, 0 tilt rate, 1.5 rad/s spin, 0␣
 ↪flap rate
init_3dof = [np.deg2rad(5), 0.0, 0.1, 0.0, 1.5, 0.0]

t_span = (0, 300)
t_eval = np.linspace(t_span[0], t_span[1], 1000)

sol = solve_ivp(ode_3dof, t_span, init_3dof, t_eval=t_eval, method='RK45')

# ==========================================
# 5. Visualizing the Baseline
# ==========================================
fig, (ax1, ax2, ax3) = plt.subplots(3, 1, figsize=(10, 8), sharex=True)

ax1.plot(sol.t, np.rad2deg(sol.y[0]), 'b')
ax1.set_ylabel('Tilt $\\theta$ (deg)')

ax2.plot(sol.t, sol.y[4], 'r')
ax2.set_ylabel('Speed $\\dot{\\psi}$ (rad/s)')
```

```
ax3.plot(sol.t, sol.y[2], 'g')
ax3.set_ylabel('Blade Flap $q$ (m)')
ax3.set_xlabel('Time (s)')

for ax in [ax1, ax2, ax3]: ax.grid(True)
plt.suptitle('3-DOF Nonlinear Baseline Simulation')
plt.tight_layout()
plt.show()
```



1.1.1 Linearize

[12]:
```
# Steady-State Operating Point

Omega = sp.Symbol('Omega', real=True, positive=True)
q0 = sp.Symbol('q0', real=True)

# Operating point
op_point = {
    theta: 0,
```

```
    theta.diff(t): 0,
    psi.diff(t): Omega,
    q: q0,
    q.diff(t): 0
}

# State Vectors
q_vec = sp.Matrix([theta, psi, q])
qd_vec = sp.Matrix([theta.diff(t),psi.diff(t),q.diff(t)])

# Linearized Matrices
forcing = LM.forcing

# Mass matrix
M_lin = sp.simplify(LM.mass_matrix.subs(op_point))
# Damping / gyroscopic matrix
C_lin = sp.simplify(-forcing.jacobian(qd_vec).subs(op_point))
# Stiffness / centrifugal matrix
K_lin = sp.simplify(-forcing.jacobian(q_vec).subs(op_point))

# Print Results
print("Linearized Matricies")
print("\nMass Matrix:")
display(M_lin)
print("\nDamping / Gyroscopic Matrix:")
display(C_lin)
print("\nStiffness / Centrifugal Matrix:")
display(K_lin)

# # Matrix Shape Check
# print("\nMatrix Dimensions:")
# print(f"M_lin: {M_lin.shape}")
# print(f"C_lin: {C_lin.shape}")
# print(f"K_lin: {K_lin.shape}")
```

```
Linearized Matricies

Mass Matrix:
```

$$\begin{bmatrix} J_{yyb}\cos^2(\psi) + J_{yyn} + J_{zzb}\sin^2(\psi) + m_b\left(X_h^2 + Z_b^2\cos^2(\psi) + 2Z_bZ_h\cos(\psi) + Z_h^2\right) + m_n\left(X_n^2 + Z_n^2\right) & X_hZ_bm_b\sin \\ X_hZ_bm_b\sin(\psi) & J_{xxb} + Z_b^2m \\ M_\phi Z_h + M_{rphi}\cos(\psi) & 0 \end{bmatrix}$$

```
Damping / Gyroscopic Matrix:
```

$$\begin{bmatrix} -J_{yyb}\Omega\sin(2\psi) + J_{zzb}\Omega\sin(2\psi) - 2\Omega Z_bm_b\left(Z_b\cos(\psi) + Z_h\right)\sin(\psi) + c_t & 2\Omega X_hZ_bm_b\cos(\psi) & -M_{rphi}\Omega\sin(\psi) \\ 0 & 0 & 0 \\ -M_{rphi}\Omega\sin(\psi) & 0 & 0 \end{bmatrix}$$

Stiffness / Centrifugal Matrix:

$$\begin{bmatrix} -Z_n g m_n - g m_b \left(Z_b \cos\left(\psi\right) + Z_h\right) + k_t & -\Omega^2 X_h Z_b m_b \sin\left(\psi\right) & 0 \\ 0 & -Z_b g m_b \cos\left(\psi\right) & 0 \\ 0 & 0 & K_b \end{bmatrix}$$

1.1.2 Eigenvalue Analysis

[13]:
```
# State-space A matrix construction

# Define the operating point dictionary
op_dict = {
    theta: 0,
    theta.diff(t): 0,
    theta.diff(t, 2): 0,
    psi.diff(t): Omega,
    psi.diff(t, 2): 0,
    q: q0,
    q.diff(t): 0,
    q.diff(t, 2): 0
}

# 2. Extract the un-inverted matrices (A_and_B=False is the default)
M_sym, A_sym, B_sym, _ = LM.linearize(
    q_ind=[theta, psi, q],
    qd_ind=[theta.diff(t), psi.diff(t), q.diff(t)],
    op_point=op_dict
)

subs_dict = {**params, psi: 0, Omega: Omega}

M_sub = M_sym.subs(subs_dict)
A_sub = A_sym.subs(subs_dict)

# 4. Create the final State-Space A matrix safely
q0_val = 0
A_state_space_3dof = sp.expand(M_sub.inv() * A_sub).evalf()

print("Flexible Blade, Tilting Nacelle State-Space A Matrix:")
display(A_state_space_3dof)
```

Flexible Blade, Tilting Nacelle State-Space A Matrix:

$$\begin{bmatrix} 0 & 0 & 0 & 1.0 & 0 \\ 0 & 0 & 0 & 0 & 1.0 \\ 0 & 0 & 0 & 0 & 0 \\ -1.46577806258678 & 0 & 16.0205062479974 & -0.0854426999893197 & 0.01025312399871840 \\ 0 & 1.20122448979592 & 0 & 0 & 0 \\ 13.741669336751 & 0 & -1400.19224607498 & 0.801025312399872 & -0.09612303748798460 \end{bmatrix}$$

[14]:
```
# ----------------------------------------------------------
# Evaluate at a specific rotor speed
# ----------------------------------------------------------
OMEGA_VAL = 1.5  # rad/s (~14 RPM)
A_evaluated = A_state_space_3dof.subs(Omega, OMEGA_VAL)

# Convert to NumPy
A_num = np.array(A_evaluated.tolist(), dtype=np.float64)

# Calculate eigenvalues and eigenvectors
eigenvalues, eigenvectors = la.eig(A_num)

# DOF index -> name mapping (positions 0-2 correspond to theta, psi, q)
dof_names = {0: "Nacelle Tilt", 1: "Rotor Azimuth", 2: "Blade Flapwise Bending"}

print("=== 3-DOF Eigenvalue Analysis Results ===")
print(f"Operating Speed: Omega = {OMEGA_VAL} rad/s\n")

for i, lam in enumerate(eigenvalues):
    # Filter numerical noise
    if np.abs(lam) < 1e-4:
        continue

    if np.abs(np.imag(lam)) > 1e-4:
        # Oscillatory mode - print positive-imaginary half only
        if np.imag(lam) > 0:
            wn_rad = np.abs(lam)
            wn_hz  = wn_rad / (2 * np.pi)
            zeta   = -np.real(lam) / wn_rad

            # Use eigenvector participation to name the mode
            dominant_dof = np.argmax(np.abs(eigenvectors[:, i])) % 3
            mode_name = dof_names.get(dominant_dof, "Unknown")

            print(f"Mode: {mode_name}")
            print(f"  Eigenvalue (λ): {np.real(lam):.4f} ± {np.imag(lam):.4f}j")
            print(f"  Natural Frequency (ω_n): {wn_hz:.4f} Hz  ({wn_rad:.4f} rad/
 ↪s)")
            print(f"  Damping Ratio (ζ): {zeta:.4f}")
            print("-" * 40)
```

```
    else:
        # Real root - only print once (positive root of the ± pair)
        if np.real(lam) > 0:
            dominant_dof = np.argmax(np.abs(eigenvectors[:, i])) % 3
            mode_name = dof_names.get(dominant_dof, "Unknown")

            print(f"Mode: {mode_name} (Non-Oscillatory)")
            print(f"  Eigenvalue (λ): {lam:.6f}")
            print(f"  Time Constant (τ): {1/np.abs(np.real(lam)):.4f} seconds")
            status = "UNSTABLE (Exponential Divergence)" if np.real(lam) > 0␣
↪else "STABLE (Exponential Decay)"
            print(f"  STATUS: {status}")
            print("-" * 40)
```

```
=== 3-DOF Eigenvalue Analysis Results ===
Operating Speed: Omega = 1.5 rad/s

Mode: Blade Flapwise Bending
  Eigenvalue (λ): -0.0046 ± 37.4212j
  Natural Frequency (ω_n): 5.9558 Hz  (37.4212 rad/s)
  Damping Ratio (ζ): 0.0001
----------------------------------------
Mode: Nacelle Tilt
  Eigenvalue (λ): -0.0381 ± 1.1432j
  Natural Frequency (ω_n): 0.1821 Hz  (1.1439 rad/s)
  Damping Ratio (ζ): 0.0333
----------------------------------------
Mode: Rotor Azimuth (Non-Oscillatory)
  Eigenvalue (λ): 1.096004+0.000000j
  Time Constant (τ): 0.9124 seconds
  STATUS: UNSTABLE (Exponential Divergence)
----------------------------------------
```

1.1.3 Simulate Linearized Model

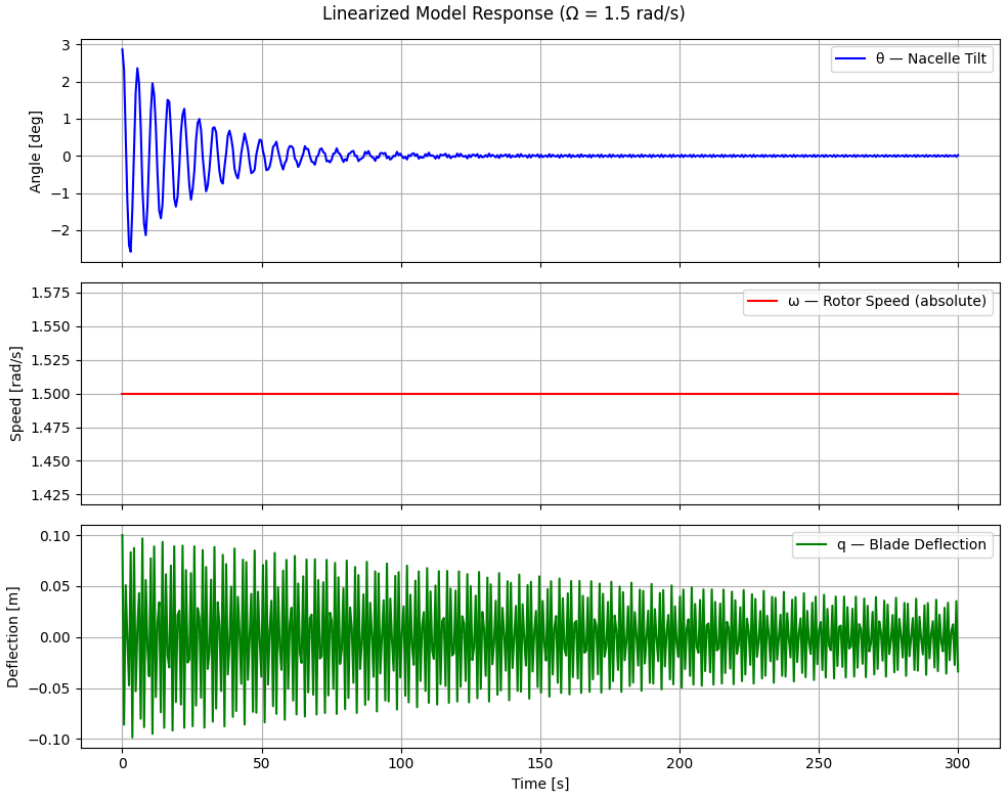
[15]:
```
# ---------------------------------------------------------
# Shared rotor speed and steady-state deflection
# ---------------------------------------------------------
OMEGA_VAL = 1.5  # rad/s - must match eigenvalue section
Q0_VAL    = 0.0  # steady-state blade deflection (m)

# Define the operating point - spinning rotor, consistent with eigenvalue section
op_dict = {
    theta: 0,
    theta.diff(t): 0,
    theta.diff(t, 2): 0,
    psi.diff(t): Omega,   # spinning, not stopped
    psi.diff(t, 2): 0,
```

```
    q: q0,
    q.diff(t): 0,
    q.diff(t, 2): 0
}

# Single unified parameter dict (same as used throughout the notebook)

# Ensure EOM are fully formed before linearizing
LM.form_lagranges_equations()

# Extract un-inverted matrices (avoids SymPy hanging on symbolic inversion)
M_sym, A_sym_uninv, B_sym, _ = LM.linearize(
    q_ind=[theta, psi, q],
    qd_ind=[theta.diff(t), psi.diff(t), q.diff(t)],
    op_point=op_dict
)

# Substitute everything: parameters + psi=0 + Omega + q0
subs_dict = {**params, psi: 0, Omega: OMEGA_VAL, q0: Q0_VAL}
M_sub = M_sym.subs(subs_dict)
A_sub = A_sym_uninv.subs(subs_dict)

# Convert to NumPy float arrays
M_num       = np.array(M_sub.evalf().tolist(), dtype=np.float64)
A_num_uninv = np.array(A_sub.evalf().tolist(), dtype=np.float64)

# Final state-space A matrix: A = M^-1 * A_uninv
A_num = la.inv(M_num) @ A_num_uninv

# ------------------------------------------------------------
# Stability analysis
# ------------------------------------------------------------
eigenvalues, eigenvectors = la.eig(A_num)

print("--- Linearized System Stability Analysis ---")
print(f"Operating Speed: Omega = {OMEGA_VAL} rad/s\n")

for i, lam in enumerate(eigenvalues):
    real_part = 0.0 if np.abs(np.real(lam)) < 1e-10 else np.real(lam)

    print(f"Mode {i+1}:")
    print(f"  Eigenvalue (λ): {lam:.4f}")
    if real_part > 0:
        print(f"  Time Constant (τ): {1/np.abs(real_part):.4f} seconds")
        print("  STATUS: UNSTABLE (Exponential Divergence)")
    elif real_part < 0:
        print(f"  Time Constant (τ): {1/np.abs(real_part):.4f} seconds")
```

```
        print("  STATUS: STABLE (Exponential Decay)")
    else:
        print("  STATUS: MARGINALLY STABLE (Oscillatory)")

# ------------------------------------------------------
# Simulate the linearized model
# ------------------------------------------------------
# x represents PERTURBATIONS from the operating point:
# x = [Δθ, Δψ, Δq, Δθ, Δψ, Δq]
x0     = np.array([0.05, 0.0, 0.1, 0.0, 0.0, 0.0])
t_span = (0, 300)
t_eval = np.linspace(t_span[0], t_span[1], 500)

def lin_sys_dynamics(t, x):
    return A_num @ x

sol = solve_ivp(lin_sys_dynamics, t_span, x0, t_eval=t_eval, method='RK45')

# ------------------------------------------------------
# Reconstruct absolute states from perturbations + operating point
# x_absolute = x_operating_point + Δx
# ------------------------------------------------------
theta_abs = sol.y[0, :]                         # op point = 0, so Δθ = θ_abs
psi_dot_abs = OMEGA_VAL + sol.y[4, :]           # Ω + Δψ (Speed perturbation is␣
 ↪index 4)
q_abs     = Q0_VAL + sol.y[2, :]                # q0 + Δq

# ------------------------------------------------------
# Plot
# ------------------------------------------------------
fig, axs = plt.subplots(3, 1, figsize=(10, 8), sharex=True)

axs[0].plot(sol.t, np.rad2deg(theta_abs), color='b', label='θ - Nacelle Tilt')
axs[0].set_ylabel('Angle [deg]')
axs[0].legend()
axs[0].grid(True)

# Updated Axis: Plotting absolute rotor speed instead of position
axs[1].plot(sol.t, psi_dot_abs, color='r', label='ω - Rotor Speed (absolute)')
axs[1].set_ylabel('Speed [rad/s]')
axs[1].legend()
axs[1].grid(True)

axs[2].plot(sol.t, q_abs, color='g', label='q - Blade Deflection')
axs[2].set_ylabel('Deflection [m]')
axs[2].set_xlabel('Time [s]')
axs[2].legend()
```

```
axs[2].grid(True)

plt.suptitle(f'Linearized Model Response (Ω = {OMEGA_VAL} rad/s)')
plt.tight_layout()
plt.show()
```

```
--- Linearized System Stability Analysis ---
Operating Speed: Omega = 1.5 rad/s

Mode 1:
  Eigenvalue (λ): -0.0046+37.4212j
  Time Constant (τ): 217.8136 seconds
  STATUS: STABLE (Exponential Decay)
Mode 2:
  Eigenvalue (λ): -0.0046-37.4212j
  Time Constant (τ): 217.8136 seconds
  STATUS: STABLE (Exponential Decay)
Mode 3:
  Eigenvalue (λ): -0.0381+1.1432j
  Time Constant (τ): 26.2259 seconds
  STATUS: STABLE (Exponential Decay)
Mode 4:
  Eigenvalue (λ): -0.0381-1.1432j
  Time Constant (τ): 26.2259 seconds
  STATUS: STABLE (Exponential Decay)
Mode 5:
  Eigenvalue (λ): 1.0960+0.0000j
  Time Constant (τ): 0.9124 seconds
  STATUS: UNSTABLE (Exponential Divergence)
Mode 6:
  Eigenvalue (λ): -1.0960+0.0000j
  Time Constant (τ): 0.9124 seconds
  STATUS: STABLE (Exponential Decay)
```



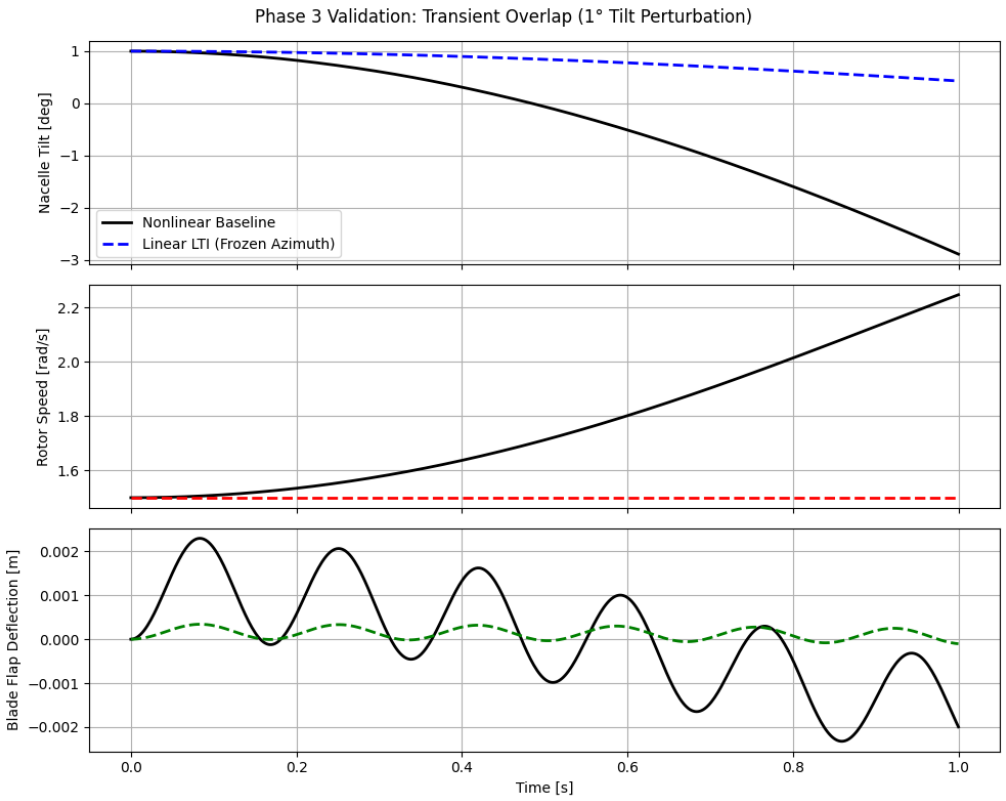
1.1.4 Validate Linearization with Integration

[16]:
```
# ======================================================================
# Phase 3: Validate Linearization via Integration (Transient Overlap)
# ======================================================================

# 1. Define the Linear ODE Function
def ode_linear_3dof(t, x_delta):
    # dx/dt = A * x
    # x_delta is the perturbation state vector:
    # [delta_theta, delta_psi, delta_q, delta_theta_dot, delta_psi_dot,␣
↪delta_q_dot]
    return A_num @ x_delta

# 2. Define Initial Conditions (1-degree tilt perturbation)
theta_pert = np.deg2rad(1.0)

# Linear Initial Conditions (Perturbations from equilibrium)
# x_delta_0 = [th, psi, q, th_d, psi_d, q_d]
```

```
init_linear = [theta_pert, 0.0, 0.0, 0.0, 0.0, 0.0]

# Nonlinear Initial Conditions (Absolute states)
# We add the perturbation to the steady-state operating point
# Assuming q0 = 0 for the steady-state deflection in this conservative model
init_nonlinear = [theta_pert, 0.0, 0.0, 0.0, OMEGA_VAL, 0.0]

# 3. Setup Simulation Time
t_span = (0, 1)  # Simulate for 1.5 seconds to capture the transient overlap and␣
 ↪divergence
t_eval = np.linspace(t_span[0], t_span[1], 500)

# 4. Integrate Both Models
print("Simulating Nonlinear Baseline...")
sol_nonlin = solve_ivp(ode_3dof, t_span, init_nonlinear, t_eval=t_eval,␣
 ↪method='RK45')

print("Simulating Linearized State-Space...")
sol_lin = solve_ivp(ode_linear_3dof, t_span, init_linear, t_eval=t_eval,␣
 ↪method='RK45')

# 5. Convert Linear Perturbations back to Absolute States for Comparison
# Total State = Operating Point + Perturbation
theta_lin_abs = np.rad2deg(sol_lin.y[0] + 0.0)
psi_d_lin_abs = sol_lin.y[4] + OMEGA_VAL
q_lin_abs     = sol_lin.y[2] + 0.0  # Add q0 here if you calculated a non-zero␣
 ↪steady-state deflection

# Convert Nonlinear States
theta_nonlin_abs = np.rad2deg(sol_nonlin.y[0])
psi_d_nonlin_abs = sol_nonlin.y[4]
q_nonlin_abs     = sol_nonlin.y[2]

# 6. Plot the Validation
fig, axs = plt.subplots(3, 1, figsize=(10, 8), sharex=True)

# Nacelle Tilt
axs[0].plot(sol_nonlin.t, theta_nonlin_abs, 'k-', lw=2, label='Nonlinear␣
 ↪Baseline')
axs[0].plot(sol_lin.t, theta_lin_abs, 'b--', lw=2, label='Linear LTI (Frozen␣
 ↪Azimuth)')
axs[0].set_ylabel('Nacelle Tilt [deg]')
axs[0].legend()

# Rotor Speed
axs[1].plot(sol_nonlin.t, psi_d_nonlin_abs, 'k-', lw=2)
```

```
axs[1].plot(sol_lin.t, psi_d_lin_abs, 'r--', lw=2)
axs[1].set_ylabel('Rotor Speed [rad/s]')

# Blade Flapwise Deflection
axs[2].plot(sol_nonlin.t, q_nonlin_abs, 'k-', lw=2)
axs[2].plot(sol_lin.t, q_lin_abs, 'g--', lw=2)
axs[2].set_ylabel('Blade Flap Deflection [m]')
axs[2].set_xlabel('Time [s]')

for ax in axs:
    ax.grid(True)

plt.suptitle('Phase 3 Validation: Transient Overlap (1° Tilt Perturbation)')
plt.tight_layout()
plt.show()
```

```
Simulating Nonlinear Baseline...
Simulating Linearized State-Space...
```



REFERENCES

[1] B. Desalegn, D. Gebeyehu, B. Tamrat, T. Tadiwose, and A. Lata, "Onshore versus offshore wind power trends and recent study practices in modeling of wind turbines' life-cycle impact assessments," *Cleaner Engineering and Technology*, vol. 17, p. 100691, 2023.

[2] M. H. Hansen, "Aeroelastic instability problems for wind turbines," *Wind Energy: An International Journal for Progress and Applications in Wind Power Conversion Technology*, vol. 10, no. 6, pp. 551–577, 2007.

[3] M. H. Hansen, "Improved modal dynamics of wind turbines to avoid stall-induced vibrations," *Wind Energy: An International Journal for Progress and Applications in Wind Power Conversion Technology*, vol. 6, no. 2, pp. 179–195, 2003.

[4] E. Branlard, "Lecture notes on wind turbine structural dynamics," tech. rep., University of Massachusetts Amherst, Amherst, MA, 2025.

[5] E. S. P. Branlard, "Flexible multibody dynamics using joint coordinates and the rayleigh-ritz approximation: The general framework behind and beyond flex," *Wind Energy*, vol. 22, no. 7, pp. 877–893, 2019.

[6] E. Branlard and J. Geisler, "A symbolic framework for flexible multibody systems applied to horizontal axis wind turbines," *Wind Energy Science Discussions*, vol. 2021, pp. 1–27, 2021.